

Artificial Intelligence Project #1 Supervised Learning Report

414551003 Yong-Cheng Liaw¹

¹National Yang Ming Chiao Tung University

1 Research Question and Motivation

Student: 414551003 Yong-Cheng Liaw

The dataset GitHub link: <https://github.com/DandinPower/prediction-market-analysis>

Prediction markets such as Polymarket [1] and Kalshi [2] have emerged as important tools for forecasting real world events. In particular, decentralized blockchain based platforms like Polymarket enable trading on a wide range of topics, including politics, economics, and entertainment, while providing transparent and publicly verifiable market data. These markets aggregate dispersed information into probabilistic forecasts through price signals driven by financial incentives [3]. Recent studies show that such markets can act as real time indicators of public sentiment and, in some cases, provide competitive or complementary forecasting performance compared to traditional polling methods [4]. This shows the strong adoption of prediction markets, as illustrated in Figure 1 from 2024 to 2026.

Polymarket: Trading Volume

Source: Blockworks

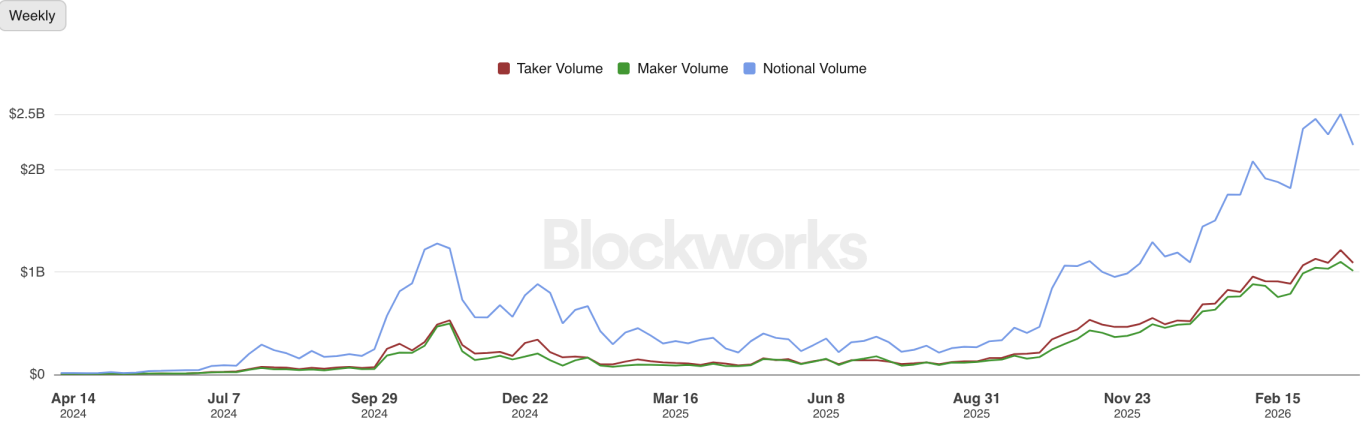


Figure 1: Polymarket trading volume from 2024 to 2026. Source: Blockworks Analysis [5].

This opens a new avenue for research. Prediction market data might contain more efficient information than raw data, since it may include insider information and public sentiment. It could provide more accurate predictions than raw data. For example, in the past we trained sequence models to predict stock market movements based on historical price data, but what about using prediction market data to predict stock market movements? Unlike raw data, which always lags behind the current time, prediction market data may include forward looking information. Therefore, using information from prediction markets, for example whether a stock will go up or down the next day, to predict stock market movements might have an advantage over using raw data such as historical price data.

Therefore, in this project, I explore the potential of using prediction market data to predict real world events by gathering data from the Polymarket official API and crawling detailed transaction data from the raw blockchain, which the official API does not provide, to train different supervised learning models on different events.

2 Dataset Introduction

The dataset used in this project is not a generic collection from the Polymarket official API. It is a reconstructed transaction-level corpus built from postprocessed Polymarket data and raw blockchain block data. After preprocessing, the datasets are stored in a standalone folder with many subfolders corresponding to each filtered market. In total, the processed corpus contains 6,010 markets, all of which were opened and settled during the short observation window from 2025-12-25 to 2025-12-31. This is clearly not a long historical panel, but it is still dense enough to expose the microstructure of active prediction markets during a period with many short-horizon contracts. The reason for this narrow time window is that the

raw blockchain data is very large and expensive to process. The public free endpoint of Polygon, the blockchain where Polymarket is built, usually has a rate limit, and since Polygon contains not only Polymarket transactions but also many other applications that run on the same blockchain, it is not possible to obtain all the data from a long time window without using a paid API or running for a very long time. Therefore, I chose to focus on a short time window that contains many active markets, which allows me to obtain a rich dataset while still being able to process it in a reasonable time frame.

The dataset for each market contains a JSON file called `metadata.json`. This file provides the surrounding context, including the market question being traded, the resolution source, the opening and closing times, the market slug, total volume, and the final resolved outcome. In other words, it tells us what the market was about and how it should ultimately be judged. The paired `yes.csv` and `no.csv` files then show how traders actually interacted with that question over time. These trade files retain the block number, timestamp, transaction hash, wallet address, buy or sell direction, token quantity, execution price, and the dollar value of each trade.

The following is an example of a transaction CSV, metadata JSON file and some overall dataset statistics.

Listing 1: Example transaction CSV used in the dataset.

```
block_number,timestamp,tx_hash,wallet,side,tokens,price,total_usdc
80826575,2025-12-26T18:06:33+00:00,0xd562783,0x06b694,BUY,29.4117,0.51,15.0
80835449,2025-12-26T23:02:21+00:00,0x4358933,0x5ad818,BUY,19.6078,0.51,10.0
80836619,2025-12-26T23:41:21+00:00,0x673d0ec,0x6258f5,BUY,2.0784,0.51,1.06
80836818,2025-12-26T23:47:59+00:00,0x1db6fd8,0x6258f5,SELL,2.07,0.5,1.035
```

Listing 2: Example market metadata JSON.

```
{
  "question": "Bitcoin Up or Down - December 26, 7PM ET",
  "resolution_source": "https://www.binance.com/en/trade/BTC_USDT",
  "startDate": "2025-12-25T00:01:14.798898Z",
  "endDate": "2025-12-27T01:00:00Z",
  "total_volume": 61513.576147,
  "outcome": "yes",
  "trade_count": 1951,
  "yes_trade_count": 995,
  "no_trade_count": 956,
  ... other metadata fields ...
}
```

Table 1: Summary statistics of the dataset.

Metric	Value
Original Markets	6011
Filtered Markets	1235
Yes Count	650
No Count	585
Yes Ratio	0.5263
No Ratio	0.4737

Table 2: Market type distribution.

Market Type	Count	Ratio
crypto_bitcoin	689	0.5579
crypto_other	520	0.4211
nfl	2	0.0016
nba	5	0.0040
cfb	6	0.0049
others	13	0.0105
Total	1235	1.0000

Table 1 shows that this dataset is relatively balanced between yes and no outcomes, with a slight skew toward yes, approximately 52.6%. No missing or unresolved outcomes remain, which is favorable for supervised learning. However, the filtering step reduces the dataset size significantly, with approximately 20.5% retained, which may impact statistical power.

Table 2 shows that this dataset is heavily dominated by crypto-related markets, approximately 97.9% combined, particularly Bitcoin. Traditional sports categories such as NFL, NBA, and CFB are almost negligible. This introduces a strong domain bias, meaning models trained on this data will likely generalize poorly outside crypto contexts unless rebalanced or augmented. The main reason the dataset is dominated by crypto-related markets is that during the short time window, a large portion of active markets on Polymarket were filtered out because I limited both the start time and end time to be

between 2025-12-25 and 2025-12-31, which is a period with many short-horizon contracts. Many diverse markets, such as politics, may run for more than one month, so they are not included in the dataset. This can be improved in the future by expanding the time window and including more markets.

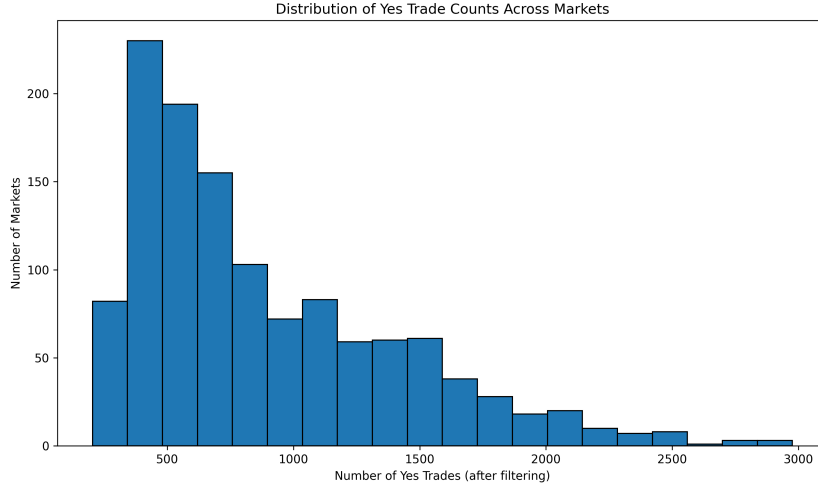


Figure 2: Distribution of trade counts across markets.

Since the dataset is reconstructed from raw blockchain data, it contains detailed transaction-level information that is not available in the official API. This allows for more granular analysis of trading behavior and market dynamics. For example, we can analyze the distribution of trade counts across markets, as shown in Figure 2. After filtering to exclude unresolved markets and those whose number of transactions per market is below 500, the distribution is highly skewed, with a few markets having a large number of transactions, while most markets have relatively few transactions, around 500 to 1000 transactions.

3 Detail Approach for Dataset Crafting and Preprocessing

3.1 Polymarket Gamma API

Polymarket exposes several public interfaces, but for dataset construction the most important one is the Gamma API, which serves as the market discovery layer of the platform [6]. The official documentation describes Gamma as the primary read-only API for browsing markets, events, and related metadata, while the `/markets` endpoint returns the detailed fields needed to identify each binary contract, including the market question, opening and closing dates, resolution source, and the CLOB token identifiers for the YES and NO sides [6, 7]. In this project, `preprocessing.fetch` uses <https://gamma-api.polymarket.com/markets> with `closed=true`, pagination through `limit` and `offset`, and date constraints on `end_date_min` and `end_date_max`. The local pipeline then applies an additional filter requiring `startDate` to be later than 2025-12-25T00:00:00Z, so the retained metadata aligns with the short observation window used for the blockchain scan.

Gamma metadata is useful because it provides semantic context that raw chain data does not contain. A Polygon transaction can tell us that a wallet traded a token, but it does not directly tell us the human-readable forecasting question or the final resolved label. The repository therefore postprocesses the Gamma response by parsing `outcomePrices` and converting fully resolved binary markets into a supervised learning label, where `["1", "0"]` is mapped to a final outcome of `yes` and `["0", "1"]` is mapped to `no`. This step transforms market metadata from a descriptive catalog into a clean label source that can later be joined with transaction-level records.

3.2 Polygon Blockchain Data and Poly-trade-scan Toolkit

The second half of the dataset comes from Polygon PoS, the EVM-compatible chain on which Polymarket trading activity is settled [8]. Polygon provides public HTTPS RPC and WebSocket endpoints for chain access, and these endpoints follow the Ethereum JSON-RPC model, where blocks contain transactions and transaction receipts contain execution logs that can be decoded into application-specific events [8, 9]. This architectural detail matters for the project because Polygon is a shared chain rather than a Polymarket-only database. A raw block range therefore contains transactions from many

unrelated decentralized applications, and Polymarket trades must be reconstructed by scanning blocks and then decoding only the fills associated with Polymarket contracts and outcome tokens.

To operationalize this step, the project relies on the open-source `poly-trade-scan` toolkit [10]. According to its documentation, the tool can either listen to Polygon blocks in real time through WebSocket or download historical trades over a specified block interval, exporting the results to CSV [10]. The preprocessing pipeline uses the historical mode, because the objective is to reconstruct a complete past window instead of monitoring new trades. The relevant block span is 80750701 through 81053200, which covers the period from 2025-12-25 to 2025-12-31, and scanning this entire window can take about one day on free RPC infrastructure. The scanner output is already close to the format needed downstream, because each row contains `block_number`, `timestamp`, `tx_hash`, `wallet`, `token_id`, `side`, `tokens`, `price`, and `total_usdc`. Among these columns, `token_id` is the critical bridge back to Gamma metadata, since it uniquely identifies which YES or NO outcome token was traded.

3.3 Complete Dataset Crafting Process

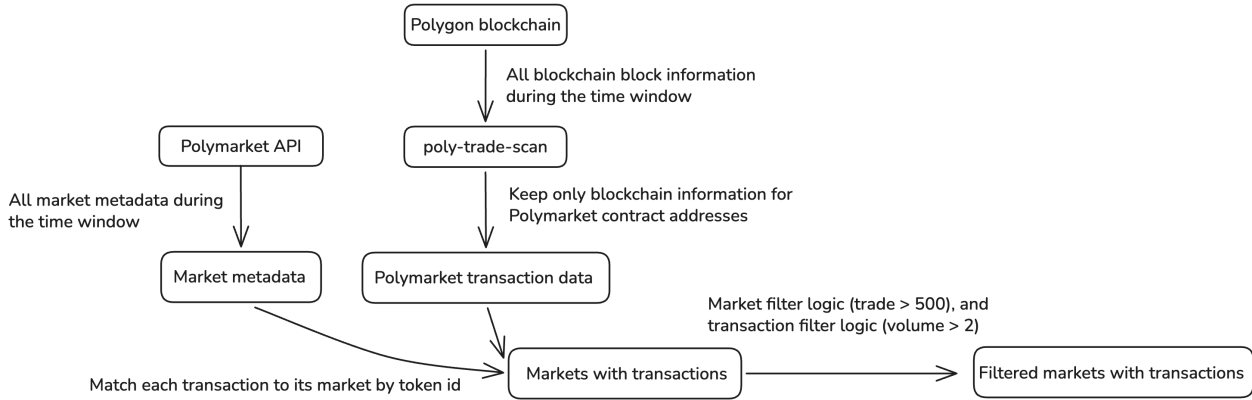


Figure 3: Overview of the dataset crafting process.

Figure 3 summarizes the full preprocessing workflow as a join between a semantic source and an execution source. The pipeline begins with market discovery by querying the Gamma `/markets` endpoint, paginating over closed markets within the target date range, and deriving the binary outcome label from `outcomePrices`. This step produces market-level metadata including `question`, `startDate`, `endDate`, `resolutionSource`, `clobTokenIds`, and the final outcome. In parallel, a chain scan is performed by downloading historical Polygon trades across blocks 80750701 to 81053200 using `poly-trade-scan`, yielding raw trade rows with `token_id`, `wallet`, `side`, `price`, `size`, and `timestamp`. The two sources are then aligned through token-based matching, where each market’s YES and NO token IDs are parsed to build a lookup table, and all trade CSV rows are streamed into their corresponding market side, producing structured outputs under `datasets/mapped_markets/<market_id>/` including `metadata.json`, `yes.csv`, and `no.csv`.

Subsequently, market-level filtering is applied to retain only mapped markets with sufficient activity, followed by a stricter selection of analysis markets satisfying `yes_trade_count > 500`, resulting in 6,010 mapped markets of which 1,235 are used in the final analysis. Trade-level filtering is then performed to extract informative YES-side trades by enforcing conditions `BUY`, `price < 0.98`, and `total_usdc > 2.0`, yielding cleaner temporal sequences suitable for visualization and downstream modeling.

The core design principle of this pipeline is that neither data source is sufficient in isolation. Gamma provides structured semantic information and resolution labels but lacks the full transaction stream, while Polygon provides detailed execution-level data without concise market interpretation. The dataset is therefore constructed by explicitly linking these two perspectives through the token identifiers in `clobTokenIds` and `token_id`. In terms of runtime characteristics, the entire process is executed on a local machine equipped with an Intel Core i3-12100 CPU and an NVIDIA RTX 2060 GPU. Fetching market metadata from Gamma takes approximately 2 minutes, downloading Polygon data for a 7-day window requires about 1 day using free RPC infrastructure, and the subsequent preprocessing and matching stages complete in roughly 5 minutes.

4 Detail Approach for Experiments

The experiments are designed to answer three closely related questions. The first question is whether early trading behavior in a prediction market already contains enough information to forecast the final resolved outcome. The second question is whether a sequential model that reads the market as a time series is better suited than a feature-based model that summarizes the same market into a fixed-length profile. The third question is whether classification performance also translates into financial usefulness once the market price at the entry point is taken into account. To make these comparisons fair, all experiments begin from the same filtered market set and the same filtered YES side trade stream. Only markets with more than 500 YES side trades are retained, and within each market only meaningful YES side buy trades are kept, excluding tiny orders and near-certain prices. This prevents the task from being dominated by low-signal noise or by trivial late-stage trades that already reveal the answer.

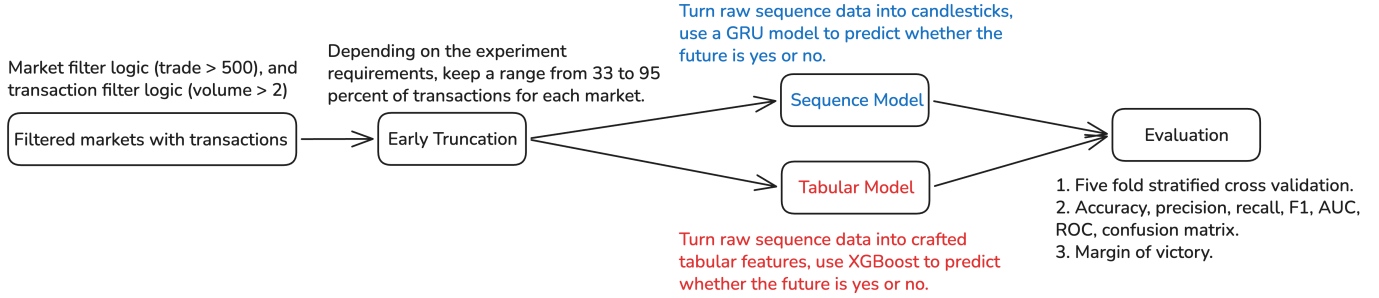


Figure 4: Overview of the experimental pipeline from filtered YES side trades to model prediction and evaluation.

The project includes five experiments with distinct purposes. Exp1 establishes the sequential baseline by applying a GRU to all filtered markets using 67% visible market history and 40 candlestick-style snapshots per market. Exp2 establishes the tabular baseline by applying XGBoost to the same market set and visibility level, using one summary profile per market. Exp3 examines how predictive performance changes as more of each market becomes visible by comparing GRU and XGBoost across visibility levels from 33% to 95%. Exp4 tests how temporal resolution affects the sequential model by varying the number of snapshots from 40 to 160 while keeping the GRU setup on all filtered markets. Exp5 tests whether a more homogeneous market family is easier to model by restricting the analysis to Bitcoin-only markets and again comparing GRU and XGBoost across visibility levels from 33% to 95%.

All experiments reported in this section were run on the same Intel Core i3-12100 plus RTX 2060 machine, and the full experiment suite takes about 30 minutes to finish.

4.1 Sequential Modeling with GRU

The first modeling path treats each market as a short financial time series and applies a gated recurrent unit to predict a yes or no outcome. Instead of raw transaction data, trades are aggregated into fixed blocks and converted into candlestick-style snapshots capturing open, close, high, low, total volume, and dollar-weighted average price. This representation stabilizes the input by preserving short-term price dynamics while reducing transaction-level noise. By default, each market is compressed into 40 snapshots, with additional experiments varying this resolution to test the impact of temporal granularity. The model is deliberately restricted to an early portion of the trade history. In the baseline, only 67% of the sequence is used, with variations ranging from 33% to 95%. This avoids trivial predictions driven by late-stage price convergence near settlement, where prices approach 1 or 0. Truncation ensures the task measures genuine early predictive signal rather than reliance on near-final outcomes.

4.2 Feature-Based Modeling with XGBoost

The second modeling path compresses each truncated market into a fixed-length feature vector and applies XGBoost for prediction. Features are grouped into four categories: price dynamics (recent level, average, range, and volatility), trading activity (total volume, trade size, count, duration, and trading speed), short-term trends (recent dollar-weighted average prices and their direction), and concentration (dominance of top wallets). Together, these features form a compact representation of price behavior, liquidity, tempo, and participant structure.

4.3 Evaluation Protocol and Economic Interpretation

Model performance is evaluated using 5-fold stratified cross-validation to preserve the balance between yes and no outcomes across splits. Within each fold, the model outputs a probability for the yes outcome, with a 50% threshold used for classification. Standard metrics are reported, including accuracy, precision, recall, F1 score, area under the receiver operating characteristic curve, log loss, and confusion matrices.

Beyond classification quality, the evaluation incorporates an economic perspective through an average margin of victory metric, which measures the expected value of purchasing a YES share at the latest observable market price prior to truncation. A YES share pays \$1 if the outcome resolves to yes and \$0 otherwise, so a trade is favorable when the model’s predicted probability exceeds the market price. For example, if the model assigns a 70% probability to a yes outcome and the entry price is \$0.60, the expected gain is $0.7 \times 0.4 - 0.3 \times 0.6 = 0.10$ dollars per trade. Positive average margins indicate systematic identification of underpriced contracts, while negative values suggest that market prices already incorporate the available signal. Entry prices are defined consistently with each modeling approach: for the sequential model, the price is the last candlestick’s dollar-weighted average price, and for the tabular model it is the most recent short-horizon dollar-weighted average price prior to truncation.

5 Experiments and Discussion

Since all runs use 5-fold stratified cross-validation, the numbers reported in this section should be read as average validation performance across the five folds rather than as the result of a single held-out test split.

5.1 Exp1: GRU model results and discussion

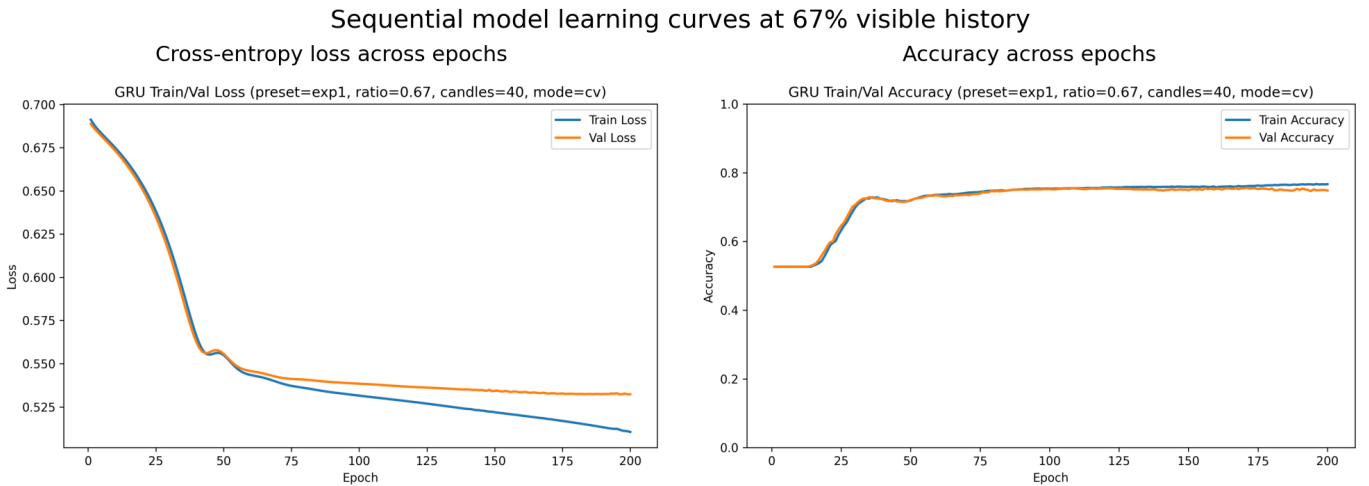


Figure 5 shows that the sequential model converges in a stable way, without the highly erratic behavior that would suggest a failed optimization process. At the default setting, where only the first 67% of each market is visible and each market is represented by 40 candlesticks, the model reaches a validation accuracy of 0.764, precision of 0.776, recall of 0.777, F1 score of 0.776, AUC of 0.807, and log loss of 0.536. In terms of the confusion matrix, it correctly identifies 505 of the 650 markets that finally resolve to yes and 439 of the 585 markets that finally resolve to no. These numbers are clearly above chance and show that early trading behavior does contain usable predictive information about the eventual resolution.

One concise example confusion matrix for the baseline GRU setting, using out-of-fold predictions from the 5-fold cross-validation, is shown below.

Table 3: Example confusion matrix for the baseline GRU model.

Actual \ Predicted	Yes	No
Yes	505	145
No	146	439

At the same time, the economic result is still negative. The average margin is -0.0521 dollars per trade, which means that even though the model classifies outcomes reasonably well, its probability estimates are not strong enough to overcome the price already implied by the market. In plain terms, the model can often guess the winner, but it usually does so at a price that is too late to create a profitable edge.

5.2 Exp2: XGBoost model results and discussion

Table 4: Baseline comparison between the sequential and feature-based models at 67% visible history.

Model	Training Accuracy	Validation Accuracy	Validation Precision	Validation Recall	Validation F1	Validation AUC	Validation Log Loss	Average Margin
GRU	0.754	0.764	0.776	0.777	0.776	0.807	0.536	-0.0521
XGBoost	0.995	0.777	0.782	0.800	0.791	0.853	0.492	-0.0539

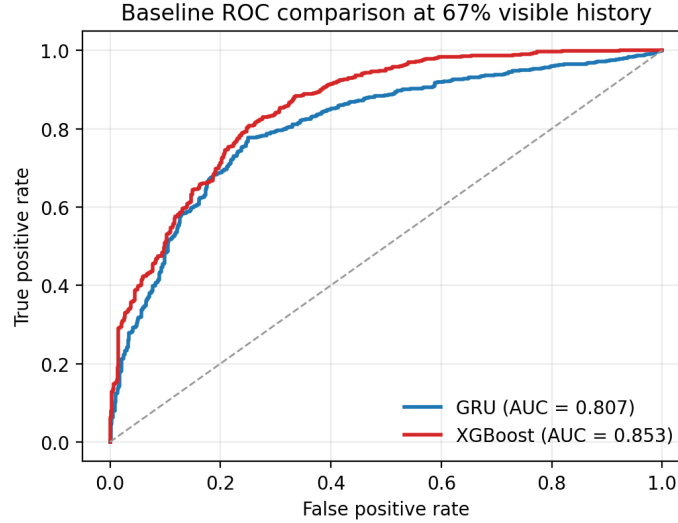


Figure 6: ROC comparison between the GRU and XGBoost baselines at 67% visible history.

Table 4 and Figure 6 show that the feature-based model is slightly stronger than the sequential model on the classification task. Its validation accuracy increases from 0.764 to 0.777, its F1 score increases from 0.776 to 0.791, and its AUC rises more clearly from 0.807 to 0.853. In confusion matrix terms, it correctly identifies 520 yes markets and 440 no markets, compared with 505 yes markets and 439 no markets for the GRU. The ROC comparison also shows a consistently better ranking ability for the feature-based model at this baseline setting.

One concise example confusion matrix for the baseline XGBoost setting, again using out-of-fold predictions from the 5-fold cross-validation, is shown below.

Table 5: Example confusion matrix for the baseline XGBoost model.

Actual \ Predicted	Yes	No
Yes	520	130
No	145	440

However, the improvement is modest on validation data and much larger on training data, where the feature-based model reaches 0.995 accuracy. This large train-to-validation gap suggests that it fits the training folds much more aggressively than the GRU. Unlike the sequential model, the feature-based model does not have epoch-by-epoch learning curves in these saved outputs, so the train-to-validation gap in Table 4 is the most informative sign of its behavior. Most importantly, the economic conclusion still does not change. Its average margin is -0.0539 dollars per trade, which is slightly worse than the GRU baseline. Therefore, the feature-based summary is better for classification, but neither baseline model produces a reliable trading advantage.

5.3 Exp3: Discussion on different truncation timings

This experiment clearly validates the expected trade-off between early prediction and late prediction. When the models only observe the first 33% of a market, accuracy is still limited, around 0.639 for the GRU and 0.623 for the feature-based

Effect of seeing more of each market before prediction

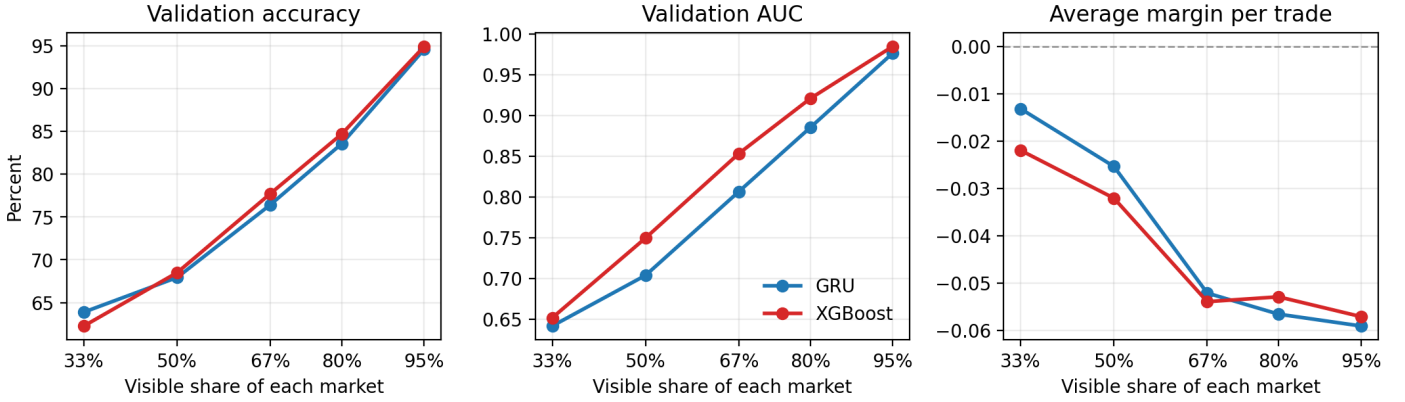


Figure 7: Validation accuracy, AUC, and average margin as the visible portion of each market increases.

Table 6: Validation results for the truncation timing sweep.

Visible Portion	GRU Accuracy	GRU AUC	GRU Margin	XGBoost Accuracy	XGBoost AUC	XGBoost Margin
33%	0.639	0.642	-0.0132	0.623	0.652	-0.0219
50%	0.679	0.704	-0.0254	0.685	0.750	-0.0320
67%	0.764	0.807	-0.0521	0.777	0.853	-0.0539
80%	0.836	0.885	-0.0566	0.847	0.921	-0.0529
95%	0.946	0.976	-0.0591	0.949	0.985	-0.0571

model, and the AUC values are only around 0.64 to 0.65. As more of the market becomes visible, both models improve steadily, and by 95% visible history they are close to perfect classification, with accuracies around 0.95 and AUC values above 0.97. This confirms that much of the predictive signal is revealed gradually as the market approaches resolution. The economic side moves in the opposite direction. The least negative average margins appear at the earliest entry point, especially for the GRU at 33%, where the loss is only about 1.3 cents per trade. As the entry point moves later, the classification problem becomes easier, but the margin becomes more negative, reaching about 0.057 and 0.059 per trade near 95%. This means the later predictions are more accurate precisely because the market price has already absorbed most of the available information. From a trading perspective, this is the key result of the project: the models become more certain as time passes, but the opportunity to exploit mispricing shrinks at the same time.

5.4 Exp4: Discussion on different candlestick construction intervals

Effect of temporal resolution in the sequential model

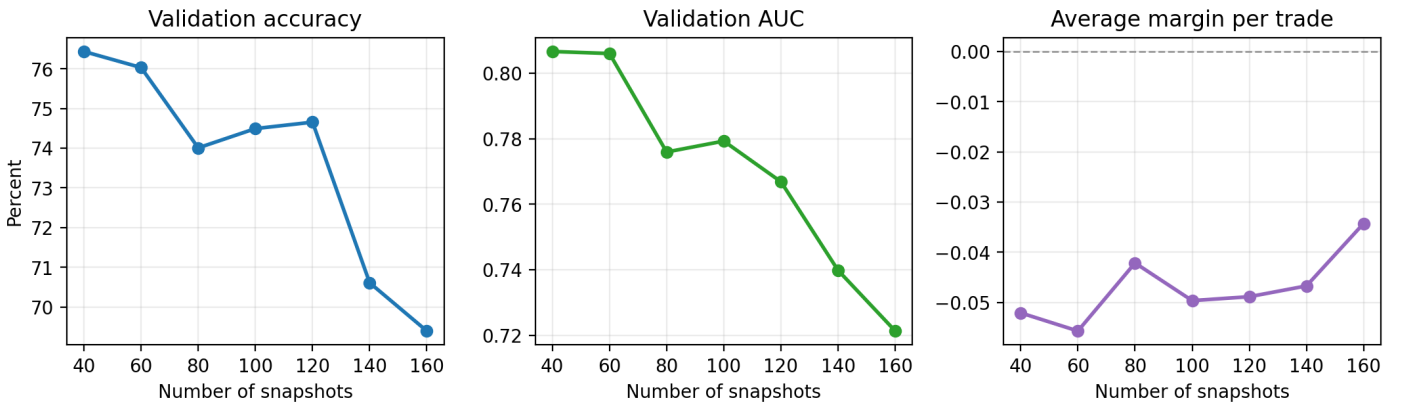


Figure 8: Validation accuracy, AUC, and average margin for different numbers of snapshots in the GRU model.

This experiment asks whether a finer time resolution helps the sequential model. If more snapshots are used, each one

contains fewer trades and can preserve more short-term detail, but it can also expose the model to more noise. The empirical result in Figure 8 is that the coarser settings work better for classification. The best validation accuracy and AUC both occur at 40 snapshots, with values of 0.764 and 0.807, while 60 snapshots remains very close. After that, performance declines gradually. At 140 snapshots, accuracy falls to 0.706 and AUC to 0.740, and at 160 snapshots it drops further to 0.694 and 0.721.

The average margin behaves somewhat differently and becomes less negative at very high resolution, reaching -0.0343 dollars per trade at 160 snapshots. However, this does not overturn the main conclusion, because classification quality deteriorates substantially and the margin still remains below zero. The more plausible interpretation is that the extra temporal detail does not add enough stable signal to justify the additional noise. For this dataset, a moderate compression of each market into about 40 to 60 snapshots is the most effective choice.

5.5 Exp5: Discussion on the Bitcoin-only scenario

Table 7: Comparison between the full market set and the Bitcoin-only subset at the common 67% setting.

Dataset Scope	Markets	Yes	No	Model	Accuracy	F1	AUC	Average Margin
All markets	1235	650	585	GRU	0.764	0.776	0.807	-0.0521
Bitcoin only	117	61	56	GRU	0.829	0.836	0.810	-0.0316
All markets	1235	650	585	XGBoost	0.777	0.791	0.853	-0.0539
Bitcoin only	117	61	56	XGBoost	0.735	0.744	0.773	-0.0765

Bitcoin-only subset versus the full market set

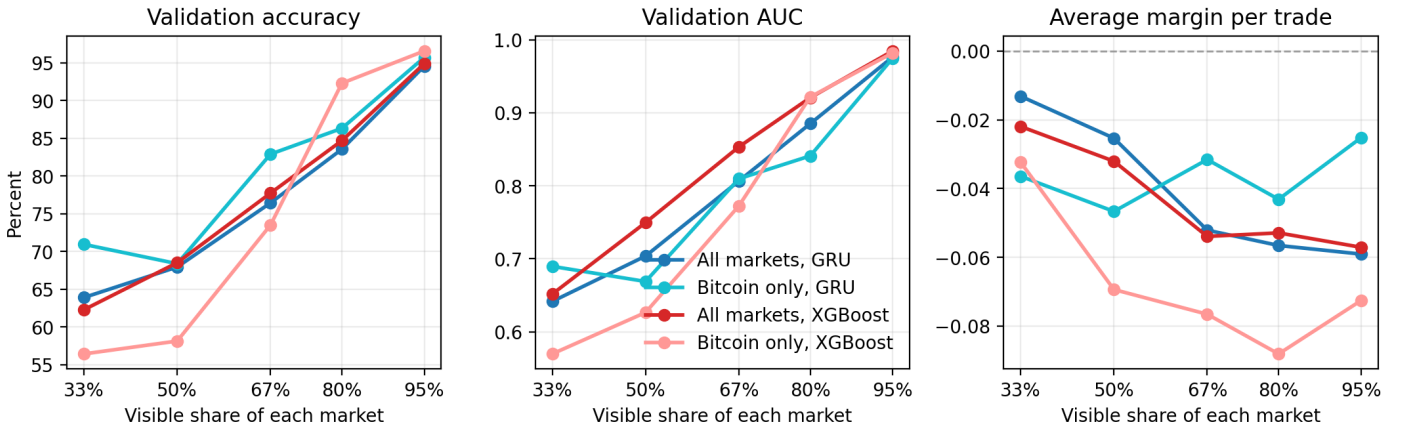


Figure 9: Full ratio sweep comparison between the all-market setting and the Bitcoin-only setting.

The Bitcoin-only subset contains 117 markets, with 61 yes outcomes and 56 no outcomes, so it is much smaller than the full dataset. Even so, Table 7 shows that the sequential model benefits noticeably from the more homogeneous market family at the standard 67% setting. Its accuracy improves from 0.764 to 0.829, its F1 score improves from 0.776 to 0.836, and its average margin improves from -0.0521 to -0.0316 dollars per trade. Figure 9 shows that this pattern largely continues across the later settings as well. At 95% visible history, the GRU reaches 0.957 accuracy and the least negative margin observed in the Bitcoin-only experiment, -0.0252 dollars per trade. This suggests that the sequential model can exploit the fact that Bitcoin contracts share a more consistent trading rhythm than the full mixture of market types.

The feature-based model does not show the same improvement. On the Bitcoin-only subset, its accuracy and AUC are actually worse than the full market results at 33%, 50%, and 67% visible history, and its average margin is more negative at every ratio. It only overtakes the full market accuracy at very late entry points such as 80% and 95%, when the classification task is already close to trivial. This pattern is consistent with stronger overfitting on the smaller subset. Therefore, the Bitcoin-only experiment does not support the simple claim that a narrower domain always helps. It helps the sequential model, but not the feature-based model, and even the better of the two still fails to produce a positive average margin.

6 Additional Discussion

With more time, two additional experiment directions would be especially valuable. First, I would expand the observation window and compare different market families, because the current dataset is dominated by crypto-related contracts and does not tell us much about whether the same patterns hold for politics, sports, or other event types. Second, I would invest more effort in the feature extraction pipeline by designing richer features, analyzing feature importance more carefully, and selecting the most useful signals for the tabular model.

The main lesson I learned is that prediction markets are predictable to some extent, but they do not look like a low-hanging fruit that can easily be turned into a money-making strategy. If the goal is only to show that supervised learning is applicable to prediction market data, that is relatively easy. If the goal is to turn this into a real trading strategy that makes money in practice, that is much harder and would require many more methods, stronger execution assumptions, and substantially deeper analysis.

The main open question is whether prediction market dynamics can actually be turned into a profitable strategy at all. Even after trying multiple supervised learning approaches and several dataset and representation variations, the average margin remains below zero, so the unresolved issue is whether the signal is fundamentally too weak after market prices are taken into account or whether a better modeling and trading design could still cross the profit line.

References

- [1] K. P. Tsang and Z. Yang, “The anatomy of polymarket: Evidence from the 2024 presidential election,” *arXiv preprint arXiv:2603.03136*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.03136>
- [2] Kalshi, “Kalshi,” <https://kalshi.com>, 2026, [Online; accessed 2026-04-03].
- [3] A. Urquhart, “Do decentralized markets beat centralized forecasters? evidence from the 2024 presidential elections,” *Al Edari*, vol. 2025, no. 171, 2025. [Online]. Available: <https://aledari.ram.gov.om/en/do-decentralized-markets-beat-centralized-forecasters-evidencefrom-the-2024-presidential-elections/>
- [4] F. Reichenbach and M. Walther, “Exploring decentralized prediction markets: Accuracy, skill, and bias on polymarket,” *SSRN Electronic Journal*, 2025. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5910522
- [5] Blockworks Research, “Polymarket: Trading volume,” <https://blockworks.com/analytics/polymarket/overview/polymarket-trading-volume>, 2026, [Online; accessed 2026-04-03].
- [6] Polymarket, “Introduction,” <https://docs.polymarket.com/api-reference>, 2026, polymarket Documentation. [Online; accessed 2026-04-03].
- [7] —, “List markets,” <https://docs.polymarket.com/api-reference/markets/list-markets>, 2026, polymarket Documentation. [Online; accessed 2026-04-03].
- [8] Polygon Labs, “Rpc endpoints,” <https://docs.polygon.technology/pos/reference/rpc-endpoints/>, 2026, [Online; accessed 2026-04-03].
- [9] ethereum.org, “Json-rpc api,” <https://ethereum.org/en/developers/docs/apis/json-rpc/>, 2025, [Online; accessed 2026-04-03].
- [10] martkir, “poly-trade-scan,” <https://github.com/martkir/poly-trade-scan>, 2026, gitHub repository. [Online; accessed 2026-04-03].

A Appendix

This appendix presents the bundled source code used for the preprocessing pipeline and the analysis and experiments modules. The full code is also available in the GitHub repository: <https://github.com/DandinPower/prediction-market-analysis>

A.1 analysis_and_experiments Package

Listing 3: Source code for the analysis_and_experiments package.

```
1 FILE: analysis_and_experiments/__init__.py
2 =====
3 """Prediction market analysis package."""
4
5 __all__ = [
6     "analysis",
7     "data",
8     "evaluation",
9     "experiments",
10    "plotting",
11    "strategies",
12 ]
13
14
15
16 =====
17 FILE: analysis_and_experiments/analysis/__init__.py
18 =====
19 """Dataset analysis modules."""
20
21 from .market_outcome import analyze_market_outcome_on_ratio
22 from .market_type import analyze_market_type
23
24 __all__ = ["analyze_market_outcome_on_ratio", "analyze_market_type"]
25
26
27
28 =====
29 FILE: analysis_and_experiments/analysis/market_outcome.py
30 =====
31 import argparse
32 from dataclasses import dataclass
33 from pathlib import Path
34 from typing import Any
35
36 from analysis_and_experiments.data import load_filtered_markets_with_trades
37
38 MAPPED_MARKETS_FOLDER_PATH = Path("datasets/mapped_markets")
39
40
41 @dataclass
42 class OutcomeAnalysisResult:
43     truncate_and_keep_ratio: float
44     total_market_folders: int
45     filtered_market_count: int
46     yes_count: int
47     no_count: int
48     unknown_count: int
49     yes_ratio: float
50     no_ratio: float
51     unknown_ratio: float
52
53
54 def _safe_ratio(count: int, total: int) -> float:
55     """Return 'count / total' or '0.0' when 'total' is not positive."""
56     if total <= 0:
57         return 0.0
58     return float(count / total)
59
60
61 def _count_outcomes(markets: list[dict[str, Any]]) -> tuple[int, int, int]:
62     """Count YES, NO, and unknown outcomes across loaded markets."""
63     yes_count = 0
```

```

64     no_count = 0
65     unknown_count = 0
66
67     for market in markets:
68         outcome = str(market.get("outcome", "")).strip().lower()
69         if outcome == "yes":
70             yes_count += 1
71         elif outcome == "no":
72             no_count += 1
73         else:
74             unknown_count += 1
75
76     return yes_count, no_count, unknown_count
77
78
79 def analyze_market_outcome_on_ratio(
80     mapped_market_folder_path: Path,
81     truncate_and_keep_ratio: float,
82 ) -> OutcomeAnalysisResult:
83     """Analyze outcome counts and ratios for a truncated mapped-market dataset."""
84     loaded_markets, _market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
85         mapped_market_folder_path,
86         truncate_and_keep_ratio,
87     )
88
89     yes_count, no_count, unknown_count = _count_outcomes(loaded_markets)
90     filtered_market_count = len(loaded_markets)
91
92     return OutcomeAnalysisResult(
93         truncate_and_keep_ratio=truncate_and_keep_ratio,
94         total_market_folders=total_market_folders,
95         filtered_market_count=filtered_market_count,
96         yes_count=yes_count,
97         no_count=no_count,
98         unknown_count=unknown_count,
99         yes_ratio=_safe_ratio(yes_count, filtered_market_count),
100        no_ratio=_safe_ratio(no_count, filtered_market_count),
101        unknown_ratio=_safe_ratio(unknown_count, filtered_market_count),
102    )
103
104
105 def _build_parser() -> argparse.ArgumentParser:
106     """Build the CLI parser for market outcome analysis."""
107     parser = argparse.ArgumentParser(
108         description=(
109             "Analyze filtered market outcomes using the same loading/filter policy as "
110             "the strategy modules."
111         )
112     )
113     parser.add_argument(
114         "--mapped-markets-folder-path",
115         type=Path,
116         default=MAPPED_MARKETS_FOLDER_PATH,
117         help=f"Path to mapped market folders (default: {MAPPED_MARKETS_FOLDER_PATH}).",
118     )
119     parser.add_argument(
120         "--truncate-and-keep-ratio",
121         type=float,
122         default=0.67,
123         help="Trade truncation ratio used by loader (default: 0.67).",
124     )
125     return parser
126
127
128 def _print_result(result: OutcomeAnalysisResult) -> None:
129     """Print a formatted summary of market outcome analysis results."""
130     print(f"\n=== truncate_and_keep_ratio={result.truncate_and_keep_ratio:.2f} ===")
131     print(f"Original market folders: {result.total_market_folders}")
132     print(f"Filtered markets: {result.filtered_market_count}")
133     print(
134         "Outcome counts | "
135         f"yes={result.yes_count}, "
136         f"no={result.no_count}, "
137         f"unknown={result.unknown_count}"
138     )

```

```

38 )
39 print(
40     "Outcome ratios | "
41     f"yes={result.yes_ratio:.4f}, "
42     f"no={result.no_ratio:.4f}, "
43     f"unknown={result.unknown_ratio:.4f}"
44 )
45
46
47 def main() -> None:
48     """Run the market outcome analysis command-line entry point."""
49     args = _build_parser().parse_args()
50     result = analyze_market_outcome_on_ratio(
51         mapped_market_folder_path=args.mapped_markets_folder_path,
52         truncate_and_keep_ratio=args.truncate_and_keep_ratio,
53     )
54     _print_result(result)
55
56
57 if __name__ == "__main__":
58     main()
59
60
61
62 =====
63 FILE: analysis_and_experiments/analysis/market_type.py
64 =====
65 import argparse
66 import json
67 import re
68 from dataclasses import dataclass
69 from pathlib import Path
70
71 from analysis_and_experiments.data import market_filter_policy
72
73 MAPPED_MARKETS_FOLDER_PATH = Path("datasets/mapped_markets")
74 MARKET_TYPES = ("crypto_bitcoin", "crypto_other", "nfl", "nba", "cfb", "others")
75
76 NFL_PATTERN = re.compile(r"\bnfl\b", flags=re.IGNORECASE)
77 NBA_PATTERN = re.compile(r"\bnba\b", flags=re.IGNORECASE)
78 CFB_PATTERN = re.compile(r"\bcfb\b", flags=re.IGNORECASE)
79 UP_PATTERN = re.compile(r"\bup\b", flags=re.IGNORECASE)
80 DOWN_PATTERN = re.compile(r"\bdown\b", flags=re.IGNORECASE)
81 BITCOIN_PATTERN = re.compile(r"\b(bitcoin|btc)\b", flags=re.IGNORECASE)
82
83
84 @dataclass
85 class MarketTypeAnalysisResult:
86     total_market_folders: int
87     filtered_market_count: int
88     type_counts: dict[str, int]
89     type_ratios: dict[str, float]
90     market_ids_by_type: dict[str, list[str]]
91
92
93 def _safe_ratio(count: int, total: int) -> float:
94     """Return 'count / total' or '0.0' when 'total' is not positive."""
95     if total <= 0:
96         return 0.0
97     return float(count / total)
98
99
100 def _load_market_metadata(market_folder: Path) -> dict[str, object]:
101     """Load 'metadata.json' for one mapped market folder."""
102     metadata_path = market_folder / "metadata.json"
103     with open(metadata_path, "r", encoding="utf-8") as file:
104         return json.load(file)
105
106
107 def _classify_market_type_from_description(description: str) -> str:
108     """Map a market description to one of the predefined market type labels."""
109     if NFL_PATTERN.search(description):
110         return "nfl"
111     if NBA_PATTERN.search(description):

```

```

112     return "nba"
113     if CFB_PATTERN.search(description):
114         return "cfb"
115     if UP_PATTERN.search(description) and DOWN_PATTERN.search(description):
116         if BITCOIN_PATTERN.search(description):
117             return "crypto_bitcoin"
118             return "crypto_other"
119     return "others"
120
121
122 def analyze_market_type(
123     mapped_market_folder_path: Path,
124     apply_market_filter: bool,
125 ) -> MarketTypeAnalysisResult:
126     """Cluster mapped markets by description with optional policy filtering."""
127     original_market_folders = list(mapped_market_folder_path.iterdir())
128     market_ids_by_type = {market_type: [] for market_type in MARKET_TYPES}
129
130     for market_folder in original_market_folders:
131         if not market_folder.is_dir():
132             continue
133
134         metadata_path = market_folder / "metadata.json"
135         if not metadata_path.exists():
136             continue
137
138         market = _load_market_metadata(market_folder)
139         if apply_market_filter and not market_filter_policy(market):
140             continue
141
142         description = str(market.get("description", ""))
143         market_type = _classify_market_type_from_description(description)
144         market_id = str(market.get("id", market_folder.name))
145         market_ids_by_type[market_type].append(market_id)
146
147     filtered_market_count = sum(len(ids) for ids in market_ids_by_type.values())
148     type_counts = {market_type: len(ids) for market_type, ids in market_ids_by_type.items()}
149     type_ratios = {
150         market_type: _safe_ratio(count, filtered_market_count)
151         for market_type, count in type_counts.items()
152     }
153
154     return MarketTypeAnalysisResult(
155         total_market_folders=len(original_market_folders),
156         filtered_market_count=filtered_market_count,
157         type_counts=type_counts,
158         type_ratios=type_ratios,
159         market_ids_by_type=market_ids_by_type,
160     )
161
162
163 def _build_parser() -> argparse.ArgumentParser:
164     """Build the CLI parser for market type analysis."""
165     parser = argparse.ArgumentParser(
166         description=(
167             "Cluster markets by description rules: nfl, nba, cfb, crypto_bitcoin "
168             "(contains both 'Up' and 'Down', and contains 'Bitcoin' or 'BTC'), "
169             "crypto_other (contains both 'Up' and 'Down' but not Bitcoin/BTC), and others."
170         )
171     )
172     parser.add_argument(
173         "--mapped-markets-folder-path",
174         type=Path,
175         default=MAPPED_MARKETS_FOLDER_PATH,
176         help=f"Path to mapped market folders (default: {MAPPED_MARKETS_FOLDER_PATH}).",
177     )
178     parser.add_argument(
179         "--disable-market-filter",
180         action="store_true",
181         help="Disable market_filter_policy. By default the same market policy is applied.",
182     )
183     parser.add_argument(
184         "--output-json-path",
185         type=Path,

```

```

286     default=None,
287     help="Optional path to save clustered market IDs as JSON.",
288 )
289 return parser
290
291
292 def _print_result(result: MarketTypeAnalysisResult) -> None:
293     """Print a formatted summary of inferred market type counts and ratios."""
294     print("\n=== Market Type Analysis ===")
295     print(f"Original market folders: {result.total_market_folders}")
296     print(f"Analyzed markets: {result.filtered_market_count}")
297     print(
298         "Type counts | "
299         f"crypto_bitcoin={result.type_counts['crypto_bitcoin']}, "
300         f"crypto_other={result.type_counts['crypto_other']}, "
301         f"nfl={result.type_counts['nfl']}, "
302         f"nba={result.type_counts['nba']}, "
303         f"cfb={result.type_counts['cfb']}, "
304         f"others={result.type_counts['others']}"
305     )
306     print(
307         "Type ratios | "
308         f"crypto_bitcoin={result.type_ratios['crypto_bitcoin']:.4f}, "
309         f"crypto_other={result.type_ratios['crypto_other']:.4f}, "
310         f"nfl={result.type_ratios['nfl']:.4f}, "
311         f"nba={result.type_ratios['nba']:.4f}, "
312         f"cfb={result.type_ratios['cfb']:.4f}, "
313         f"others={result.type_ratios['others']:.4f}"
314     )
315
316
317 def _save_clusters(output_json_path: Path, result: MarketTypeAnalysisResult) -> None:
318     """Persist clustered market IDs grouped by inferred market type."""
319     output_json_path.parent.mkdir(parents=True, exist_ok=True)
320     with open(output_json_path, "w", encoding="utf-8") as file:
321         json.dump(result.market_ids_by_type, file, indent=2)
322     print(f"Saved clusters to: {output_json_path}")
323
324
325 def main() -> None:
326     """Run the market type analysis command-line entry point."""
327     args = _build_parser().parse_args()
328     result = analyze_market_type(
329         mapped_market_folder_path=args.mapped_markets_folder_path,
330         apply_market_filter=not args.disable_market_filter,
331     )
332     _print_result(result)
333
334     if args.output_json_path is not None:
335         _save_clusters(args.output_json_path, result)
336
337
338 if __name__ == "__main__":
339     main()
340
341
342
343 =====
344 FILE: analysis_and_experiments/data/__init__.py
345 =====
346 """Data loading and filtering policies."""
347
348 from .loader import load_filtered_markets_with_trades, load_market_and_trades
349 from .policies import btc_market_filter_policy, market_filter_policy, trade_filter_policy
350
351 __all__ = [
352     "btc_market_filter_policy",
353     "load_filtered_markets_with_trades",
354     "load_market_and_trades",
355     "market_filter_policy",
356     "trade_filter_policy",
357 ]
358
359

```

```

360
361 =====
362 FILE: analysis_and_experiments/data/loader.py
363 =====
364 import csv
365 import json
366 from pathlib import Path
367 from typing import Any, Callable
368
369 from .policies import market_filter_policy, trade_filter_policy
370
371 TradePolicy = Callable[[dict[str, Any]], bool]
372 MarketPolicy = Callable[[dict[str, Any]], bool]
373
374
375 def load_market_and_trades(
376     market_folder: Path,
377     truncate_and_keep_ratio: float,
378     *,
379     trade_policy: TradePolicy = trade_filter_policy,
380 ) -> tuple[dict[str, Any], list[dict[str, Any]]]:
381     """Load market metadata and filtered YES trades from one mapped market folder."""
382     if not (0.0 < truncate_and_keep_ratio <= 1.0):
383         raise ValueError("truncate_and_keep_ratio must be in (0, 1].")
384
385     market_metadata_path = market_folder / "metadata.json"
386     with open(market_metadata_path, "r", encoding="utf-8") as file:
387         market = json.load(file)
388
389     yes_trades_path = market_folder / "yes.csv"
390     yes_trades: list[dict[str, Any]] = []
391     with open(yes_trades_path, "r", encoding="utf-8") as csvfile:
392         csv_reader = csv.DictReader(csvfile)
393         for row in csv_reader:
394             if trade_policy(row):
395                 yes_trades.append(row)
396
397     truncate_length = int(len(yes_trades) * truncate_and_keep_ratio)
398     yes_trades = yes_trades[:truncate_length]
399
400     return market, yes_trades
401
402
403 def load_filtered_markets_with_trades(
404     mapped_market_folder_path: Path,
405     truncate_and_keep_ratio: float,
406     *,
407     market_policy: MarketPolicy = market_filter_policy,
408     trade_policy: TradePolicy = trade_filter_policy,
409 ) -> tuple[list[dict[str, Any]], dict[str, list[dict[str, Any]]], int]:
410     """Load mapped markets and return (filtered markets, market_id->trades, total folders)."""
411     loaded_markets: list[dict[str, Any]] = []
412     market_id_to_trades: dict[str, list[dict[str, Any]]] = {}
413
414     original_market_folders = list(mapped_market_folder_path.iterdir())
415     for market_folder in original_market_folders:
416         if not market_folder.is_dir():
417             continue
418
419         market, yes_trades = load_market_and_trades(
420             market_folder,
421             truncate_and_keep_ratio,
422             trade_policy=trade_policy,
423         )
424         if market_policy(market):
425             market_id = str(market["id"])
426             loaded_markets.append(market)
427             market_id_to_trades[market_id] = yes_trades
428
429     return loaded_markets, market_id_to_trades, len(original_market_folders)
430
431
432
433 =====

```

```

434 FILE: analysis_and_experiments/data/policies.py
435 =====
436 from typing import Any
437
438
439 def market_filter_policy(market: dict[str, Any]) -> bool:
440     """Keep markets with enough trade activity."""
441     return int(market.get("yes_trade_count", 0)) > 500
442
443
444 def btc_market_filter_policy(market: dict[str, Any]) -> bool:
445     """Keep active BTC/USDT markets with enough YES-side trade activity."""
446     if not market_filter_policy(market):
447         return False
448     description = str(market.get("description", ""))
449     return "btc/usdt" in description.lower()
450
451
452 def trade_filter_policy(trade: dict[str, Any]) -> bool:
453     """Keep meaningful BUY trades below near-certain pricing."""
454     return (
455         str(trade.get("side", "")) == "BUY"
456         and float(trade.get("price", 0.0)) < 0.98
457         and float(trade.get("total_usdc", 0.0)) > 2.0
458     )
459
460
461
462 =====
463 FILE: analysis_and_experiments/evaluation/__init__.py
464 =====
465 """Model evaluation utilities."""
466
467 from .margin import calculate_average_margin_of_victory
468 from .metrics import evaluate_binary_metrics
469
470 __all__ = ["calculate_average_margin_of_victory", "evaluate_binary_metrics"]
471
472
473
474 =====
475 FILE: analysis_and_experiments/evaluation/margin.py
476 =====
477 from typing import Sequence
478
479
480 def calculate_average_margin_of_victory(
481     probabilities: Sequence[float],
482     reference_prices: Sequence[float],
483 ) -> float:
484     """Calculate average expected margin using model probability and reference entry price."""
485     if len(probabilities) != len(reference_prices):
486         raise ValueError("probabilities and reference_prices must have the same length.")
487     if not probabilities:
488         raise ValueError("probabilities cannot be empty.")
489
490     margin_of_victory = 0.0
491     for probability, price in zip(probabilities, reference_prices, strict=True):
492         winning_margin = 1.0 - float(price)
493         expected_margin = float(probability) * winning_margin
494         expected_loss = (1.0 - float(probability)) * float(price)
495         margin_of_victory += expected_margin - expected_loss
496
497     return float(margin_of_victory / len(reference_prices))
498
499
500
501 =====
502 FILE: analysis_and_experiments/evaluation/metrics.py
503 =====
504 from typing import Any, Sequence
505
506 import numpy as np
507 import torch

```

```

308 from sklearn.metrics import (
309     accuracy_score,
310     f1_score,
311     log_loss,
312     precision_score,
313     recall_score,
314     roc_auc_score,
315     roc_curve,
316 )
317
318 from analysis_and_experiments.strategies.common import BinaryMetrics
319
320
321 def _to_numpy_1d(tensor: torch.Tensor) -> np.ndarray:
322     """Detach a column tensor and return it as a squeezed NumPy array."""
323     return tensor.detach().cpu().squeeze(1).numpy()
324
325
326 def evaluate_binary_metrics(
327     y_true_tensor: torch.Tensor,
328     y_prob: Sequence[float] | np.ndarray,
329     *,
330     classification_threshold: float = 0.5,
331 ) -> BinaryMetrics:
332     """Compute binary metrics and ROC data from true labels and probabilities."""
333     y_true = _to_numpy_1d(y_true_tensor)
334     y_prob_np = np.asarray(list(y_prob), dtype=np.float64)
335     y_prob_clipped = np.clip(y_prob_np, 1e-9, 1.0 - 1e-9)
336     y_pred = (y_prob_np >= classification_threshold).astype(int)
337
338     sklearn_loss = float(log_loss(y_true, y_prob_clipped, labels=[0, 1]))
339     torch_loss = float(
340         torch.nn.BCELoss()(
341             torch.tensor(y_prob_clipped, dtype=torch.float32).unsqueeze(1),
342             y_true_tensor.float(),
343         ).item()
344     )
345
346     accuracy = float(accuracy_score(y_true, y_pred))
347     precision = float(precision_score(y_true, y_pred, zero_division=0))
348     recall = float(recall_score(y_true, y_pred, zero_division=0))
349     f1 = float(f1_score(y_true, y_pred, zero_division=0))
350
351     auc: float | None
352     roc_fpr: list[float] | None
353     roc_tpr: list[float] | None
354     if len(set(y_true.tolist())) < 2:
355         auc = None
356         roc_fpr = None
357         roc_tpr = None
358     else:
359         auc = float(roc_auc_score(y_true, y_prob_np))
360         fpr, tpr, _ = roc_curve(y_true, y_prob_np)
361         roc_fpr = [float(value) for value in fpr]
362         roc_tpr = [float(value) for value in tpr]
363
364     return BinaryMetrics(
365         sklearn_log_loss=sklearn_loss,
366         torch_bce_loss=torch_loss,
367         accuracy=accuracy,
368         precision=precision,
369         recall=recall,
370         f1=f1,
371         auc=auc,
372         roc_fpr=roc_fpr,
373         roc_tpr=roc_tpr,
374     )
375
376
377 def to_numpy_1d(tensor: torch.Tensor) -> Any:
378     """Compatibility helper for legacy wrappers."""
379     return _to_numpy_1d(tensor)
380
381

```

```

382
383 =====
384 FILE: analysis_and_experiments/experiments/__init__.py
385 =====
386 """Experiment presets and runner."""
387
388 from .runner import main, run_preset
389
390 __all__ = ["main", "run_preset"]
391
392
393
394 =====
395 FILE: analysis_and_experiments/experiments/io.py
396 =====
397 import csv
398 import json
399 from pathlib import Path
400 from typing import Any
401
402 from analysis_and_experiments.strategies.common import ExperimentSummary, FoldResult, RunResult
403
404
405 def _run_to_flat_row(run: RunResult) -> dict[str, Any]:
406     """Flatten a run result into a single summary row for CSV export."""
407     return {
408         "preset": run.preset,
409         "strategy": run.strategy,
410         "truncate_and_keep_ratio": run.truncate_and_keep_ratio,
411         "desired_num_candlesticks": run.desired_num_candlesticks,
412         "cv_enabled": run.cv_enabled,
413         "cv_folds": run.cv_folds,
414         "train_log_loss": run.train_metrics.sklearn_log_loss,
415         "train_bce_loss": run.train_metrics.torch_bce_loss,
416         "train_accuracy": run.train_metrics.accuracy,
417         "train_precision": run.train_metrics.precision,
418         "train_recall": run.train_metrics.recall,
419         "train_f1": run.train_metrics.f1,
420         "train_auc": run.train_metrics.auc,
421         "val_log_loss": run.val_metrics.sklearn_log_loss,
422         "val_bce_loss": run.val_metrics.torch_bce_loss,
423         "val_accuracy": run.val_metrics.accuracy,
424         "val_precision": run.val_metrics.precision,
425         "val_recall": run.val_metrics.recall,
426         "val_f1": run.val_metrics.f1,
427         "val_auc": run.val_metrics.auc,
428         "average_margin_of_victory": run.average_margin_of_victory,
429         "roc_plot_path": str(run.roc_plot_path) if run.roc_plot_path is not None else None,
430         "gru_train_curve_plot_path": (
431             str(run.gru_train_curve_plot_path) if run.gru_train_curve_plot_path is not None else None
432         ),
433         "gru_val_curve_plot_path": (
434             str(run.gru_val_curve_plot_path) if run.gru_val_curve_plot_path is not None else None
435         ),
436         "train_confusion_matrix_plot_path": (
437             str(run.train_confusion_matrix_plot_path)
438             if run.train_confusion_matrix_plot_path is not None
439             else None
440         ),
441         "val_confusion_matrix_plot_path": (
442             str(run.val_confusion_matrix_plot_path) if run.val_confusion_matrix_plot_path is not None else None
443         ),
444     }
445
446
447 def _fold_to_flat_row(run: RunResult, fold: FoldResult) -> dict[str, Any]:
448     """Flatten one fold result into a single summary row for CSV export."""
449     return {
450         "preset": run.preset,
451         "strategy": run.strategy,
452         "truncate_and_keep_ratio": run.truncate_and_keep_ratio,
453         "desired_num_candlesticks": run.desired_num_candlesticks,
454         "cv_enabled": run.cv_enabled,
455         "cv_folds": run.cv_folds,

```

```

656         "fold_index": fold.fold_index,
657         "train_size": fold.train_size,
658         "val_size": fold.val_size,
659         "train_log_loss": fold.train_metrics.sklearn_log_loss,
660         "train_bce_loss": fold.train_metrics.torch_bce_loss,
661         "train_accuracy": fold.train_metrics.accuracy,
662         "train_precision": fold.train_metrics.precision,
663         "train_recall": fold.train_metrics.recall,
664         "train_f1": fold.train_metrics.f1,
665         "train_auc": fold.train_metrics.auc,
666         "val_log_loss": fold.val_metrics.sklearn_log_loss,
667         "val_bce_loss": fold.val_metrics.torch_bce_loss,
668         "val_accuracy": fold.val_metrics.accuracy,
669         "val_precision": fold.val_metrics.precision,
670         "val_recall": fold.val_metrics.recall,
671         "val_f1": fold.val_metrics.f1,
672         "val_auc": fold.val_metrics.auc,
673         "average_margin_of_victory": fold.average_margin_of_victory,
674     }
675
676
677 def write_summary_outputs(
678     summary: ExperimentSummary,
679     output_root: Path,
680 ) -> tuple[Path, Path, Path]:
681     """Write experiment summaries to CSV and JSON output files."""
682     result_dir = output_root / "results" / summary.preset
683     result_dir.mkdir(parents=True, exist_ok=True)
684
685     csv_path = result_dir / "summary.csv"
686     folds_csv_path = result_dir / "summary_folds.csv"
687     json_path = result_dir / "summary.json"
688
689     rows = [_run_to_flat_row(run) for run in summary.runs]
690     fold_rows = [
691         _fold_to_flat_row(run, fold)
692         for run in summary.runs
693         for fold in run.fold_results
694     ]
695
696     if rows:
697         with open(csv_path, "w", encoding="utf-8", newline="") as csvfile:
698             writer = csv.DictWriter(csvfile, fieldnames=list(rows[0].keys()))
699             writer.writeheader()
700             writer.writerows(rows)
701     else:
702         with open(csv_path, "w", encoding="utf-8") as csvfile:
703             csvfile.write("\n")
704
705     if fold_rows:
706         with open(folds_csv_path, "w", encoding="utf-8", newline="") as csvfile:
707             writer = csv.DictWriter(csvfile, fieldnames=list(fold_rows[0].keys()))
708             writer.writeheader()
709             writer.writerows(fold_rows)
710     else:
711         with open(folds_csv_path, "w", encoding="utf-8") as csvfile:
712             csvfile.write("\n")
713
714     payload = {
715         "preset": summary.preset,
716         "runs": [run.to_dict() for run in summary.runs],
717         "ranking_by_margin": summary.ranking_by_margin,
718         "ranking_by_auc": summary.ranking_by_auc,
719     }
720
721     with open(json_path, "w", encoding="utf-8") as jsonfile:
722         json.dump(payload, jsonfile, indent=2)
723
724     return csv_path, folds_csv_path, json_path
725
726
727
728 =====
729 FILE: analysis_and_experiments/experiments/presets.py

```

```

730 =====
731 from dataclasses import dataclass
732
733 DEFAULT_RATIO_SWEEP = [0.33, 0.50, 0.67, 0.80, 0.95]
734 DEFAULT_GRU_CANDLES = 40
735 DEFAULT_EXP4_CANDLE_SWEEP = [40, 60, 80, 100, 120, 140, 160]
736 PRESET_ORDER = ["exp1", "exp2", "exp3", "exp4", "exp5"]
737
738
739 @dataclass
740 class ExperimentTask:
741     preset: str
742     strategy: str
743     truncate_and_keep_ratio: float
744     desired_num_candlesticks: int | None
745
746
747 def _validate_ratios(ratios: list[float]) -> None:
748     """Validate that ratio overrides are present and fall within '(0, 1]'."""
749     if not ratios:
750         raise ValueError("ratios cannot be empty.")
751     for ratio in ratios:
752         if ratio <= 0.0 or ratio > 1.0:
753             raise ValueError(f"ratio must be in (0, 1], got {ratio}.")
754
755
756 def _validate_candles(candles: list[int]) -> None:
757     """Validate that GRU candlestick counts are present and strictly positive."""
758     if not candles:
759         raise ValueError("gru candle list cannot be empty.")
760     for candle in candles:
761         if candle <= 0:
762             raise ValueError(f"candle count must be > 0, got {candle}.")
763
764
765 def build_preset_tasks(
766     preset: str,
767     *,
768     ratios_override: list[float] | None = None,
769     gru_candles_override: list[int] | None = None,
770 ) -> list[ExperimentTask]:
771     """Build the ordered experiment tasks required for a named preset."""
772     if preset not in PRESET_ORDER:
773         raise ValueError(f"Unsupported preset: {preset}")
774
775     ratios = ratios_override if ratios_override is not None else DEFAULT_RATIO_SWEEP
776     _validate_ratios(ratios)
777
778     default_candles = gru_candles_override[0] if gru_candles_override else DEFAULT_GRU_CANDLES
779     if default_candles <= 0:
780         raise ValueError("gru candle count must be > 0.")
781
782     tasks: list[ExperimentTask] = []
783
784     if preset == "exp1":
785         tasks.append(
786             ExperimentTask(
787                 preset="exp1",
788                 strategy="gru",
789                 truncate_and_keep_ratio=ratios[0] if ratios_override else 0.67,
790                 desired_num_candlesticks=default_candles,
791             )
792         )
793
794     elif preset == "exp2":
795         tasks.append(
796             ExperimentTask(
797                 preset="exp2",
798                 strategy="tabular",
799                 truncate_and_keep_ratio=ratios[0] if ratios_override else 0.67,
800                 desired_num_candlesticks=None,
801             )
802         )
803

```

```

$04 elif preset in {"exp3", "exp5"}:
$05     ratio_sweep = ratios_override if ratios_override else DEFAULT_RATIO_SWEEP
$06     for ratio in ratio_sweep:
$07         tasks.append(
$08             ExperimentTask(
$09                 preset=preset,
$10                 strategy="gru",
$11                 truncate_and_keep_ratio=ratio,
$12                 desired_num_candlesticks=default_candles,
$13             )
$14         )
$15     tasks.append(
$16         ExperimentTask(
$17             preset=preset,
$18             strategy="tabular",
$19             truncate_and_keep_ratio=ratio,
$20             desired_num_candlesticks=None,
$21         )
$22     )
$23
$24 elif preset == "exp4":
$25     candle_sweep = gru_candles_override if gru_candles_override else DEFAULT_EXP4_CANDLE_SWEEP
$26     _validate_candles(candle_sweep)
$27     ratio = ratios[0] if ratios_override else 0.67
$28     for candle_count in candle_sweep:
$29         tasks.append(
$30             ExperimentTask(
$31                 preset="exp4",
$32                 strategy="gru",
$33                 truncate_and_keep_ratio=ratio,
$34                 desired_num_candlesticks=candle_count,
$35             )
$36         )
$37
$38     return tasks
$39
$40
$41
$42 =====
$43 FILE: analysis_and_experiments/experiments/runner.py
$44 =====
$45 import argparse
$46 from pathlib import Path
$47 from typing import Any, Callable
$48
$49 from analysis_and_experiments.data import btc_market_filter_policy, market_filter_policy
$50 from analysis_and_experiments.experiments.io import write_summary_outputs
$51 from analysis_and_experiments.experiments.presets import PRESET_ORDER, build_preset_tasks
$52 from analysis_and_experiments.strategies import (
$53     ExperimentSummary,
$54     RunResult,
$55     run_gru_experiment_on_ratio,
$56     run_tabular_experiment_on_ratio,
$57 )
$58
$59
$60 def _parse_csv_floats(raw: str | None) -> list[float] | None:
$61     """Parse a comma-separated list of floats from CLI input."""
$62     if raw is None:
$63         return None
$64     values = [chunk.strip() for chunk in raw.split(",") if chunk.strip()]
$65     if not values:
$66         return None
$67     return [float(value) for value in values]
$68
$69
$70 def _parse_csv_ints(raw: str | None) -> list[int] | None:
$71     """Parse a comma-separated list of integers from CLI input."""
$72     if raw is None:
$73         return None
$74     values = [chunk.strip() for chunk in raw.split(",") if chunk.strip()]
$75     if not values:
$76         return None
$77     return [int(value) for value in values]

```

```

878
879
880 def _select_market_policy_for_preset(preset: str) -> Callable[[dict[str, Any]], bool]:
881     """Return the market filter policy associated with a preset name."""
882     if preset == "exp5":
883         return btc_market_filter_policy
884     return market_filter_policy
885
886
887 def _build_rankings(runs: list[RunResult]) -> tuple[list[dict[str, object]], list[dict[str, object]]]:
888     """Build rankings ordered by expected margin and validation AUC."""
889     ranking_by_margin = []
890     for rank, run in enumerate(
891         sorted(runs, key=lambda item: item.average_margin_of_victory, reverse=True),
892         start=1,
893     ):
894         ranking_by_margin.append(
895             {
896                 "rank": rank,
897                 "strategy": run.strategy,
898                 "truncate_and_keep_ratio": run.truncate_and_keep_ratio,
899                 "desired_num_candlesticks": run.desired_num_candlesticks,
900                 "val_auc": run.val_metrics.auc,
901                 "average_margin_of_victory": run.average_margin_of_victory,
902             }
903         )
904
905     ranking_by_auc = []
906     for rank, run in enumerate(
907         sorted(
908             runs,
909             key=lambda item: item.val_metrics.auc if item.val_metrics.auc is not None else float("-inf"),
910             reverse=True,
911         ),
912         start=1,
913     ):
914         ranking_by_auc.append(
915             {
916                 "rank": rank,
917                 "strategy": run.strategy,
918                 "truncate_and_keep_ratio": run.truncate_and_keep_ratio,
919                 "desired_num_candlesticks": run.desired_num_candlesticks,
920                 "val_auc": run.val_metrics.auc,
921                 "average_margin_of_victory": run.average_margin_of_victory,
922             }
923         )
924
925     return ranking_by_margin, ranking_by_auc
926
927
928 def _print_run_result(run: RunResult) -> None:
929     """Print a formatted summary of one completed experiment run."""
930     train_auc_text = f"{run.train_metrics.auc:.4f}" if run.train_metrics.auc is not None else "N/A"
931     val_auc_text = f"{run.val_metrics.auc:.4f}" if run.val_metrics.auc is not None else "N/A"
932     mode_text = f"cv={run.cv_folds}-fold" if run.cv_enabled else "single-split"
933     print(
934         f"[{run.preset}] strategy={run.strategy}, ratio={run.truncate_and_keep_ratio:.2f}, "
935         f"candles={run.desired_num_candlesticks}, mode={mode_text}"
936     )
937     print(
938         "Train | "
939         f"log_loss={run.train_metrics.sklearn_log_loss:.6f}, "
940         f"bce_loss={run.train_metrics.torch_bce_loss:.6f}, "
941         f"acc={run.train_metrics.accuracy:.4f}, "
942         f"precision={run.train_metrics.precision:.4f}, "
943         f"recall={run.train_metrics.recall:.4f}, "
944         f"f1={run.train_metrics.f1:.4f}, "
945         f"auc={train_auc_text}"
946     )
947     print(
948         "Val | "
949         f"log_loss={run.val_metrics.sklearn_log_loss:.6f}, "
950         f"bce_loss={run.val_metrics.torch_bce_loss:.6f}, "
951         f"acc={run.val_metrics.accuracy:.4f}, "

```

```

952     f"precision={run.val_metrics.precision:.4f}, "
953     f"recall={run.val_metrics.recall:.4f}, "
954     f"f1={run.val_metrics.f1:.4f}, "
955     f"auc={val_auc_text}, "
956     f"margin={run.average_margin_of_victory:.6f}"
957 )
958 if run.roc_plot_path is not None:
959     print(f"ROC plot saved: {run.roc_plot_path}")
960 else:
961     print("ROC plot skipped: validation labels contain a single class.")
962 if run.gru_train_curve_plot_path is not None:
963     print(f"GRU train/val loss plot saved: {run.gru_train_curve_plot_path}")
964 if run.gru_val_curve_plot_path is not None:
965     print(f"GRU train/val accuracy plot saved: {run.gru_val_curve_plot_path}")
966 if run.train_confusion_matrix_plot_path is not None:
967     print(f"Train confusion matrix saved: {run.train_confusion_matrix_plot_path}")
968 if run.val_confusion_matrix_plot_path is not None:
969     print(f"Val confusion matrix saved: {run.val_confusion_matrix_plot_path}")
970 if run.fold_results:
971     print(f"Recorded {len(run.fold_results)} fold metric rows.")
972
973
974 def run_preset(
975     preset: str,
976     *,
977     mapped_markets_path: Path,
978     output_root: Path,
979     ratios_override: list[float] | None = None,
980     gru_candles_override: list[int] | None = None,
981     cv_folds: int = 5,
982     disable_cv: bool = False,
983 ) -> ExperimentSummary:
984     """Execute all tasks for a preset and persist the aggregated outputs."""
985     tasks = build_preset_tasks(
986         preset,
987         ratios_override=ratios_override,
988         gru_candles_override=gru_candles_override,
989     )
990     market_policy = _select_market_policy_for_preset(preset)
991
992     print(f"Experiment artifacts are written under: {output_root}")
993
994     runs: list[RunResult] = []
995     for task in tasks:
996         print(
997             f"\nRunning task: preset={task.preset}, strategy={task.strategy}, "
998             f"ratio={task.truncate_and_keep_ratio:.2f}, candles={task.desired_num_candlesticks}"
999         )
1000
1001         if task.strategy == "gru":
1002             result = run_gru_experiment_on_ratio(
1003                 mapped_markets_path,
1004                 task.truncate_and_keep_ratio,
1005                 preset=task.preset,
1006                 desired_num_candlesticks=task.desired_num_candlesticks or 40,
1007                 market_policy=market_policy,
1008                 use_cross_validation=not disable_cv,
1009                 cv_folds=cv_folds,
1010                 roc_output_dir=output_root / "plots" / "roc" / "gru",
1011                 train_val_curves_output_dir=output_root / "plots" / "training_curves" / "gru",
1012                 confusion_matrix_output_dir=output_root / "plots" / "confusion_matrix" / "gru",
1013             )
1014         elif task.strategy == "tabular":
1015             result = run_tabular_experiment_on_ratio(
1016                 mapped_markets_path,
1017                 task.truncate_and_keep_ratio,
1018                 preset=task.preset,
1019                 market_policy=market_policy,
1020                 use_cross_validation=not disable_cv,
1021                 cv_folds=cv_folds,
1022                 roc_output_dir=output_root / "plots" / "roc" / "tabular",
1023                 confusion_matrix_output_dir=output_root / "plots" / "confusion_matrix" / "tabular",
1024             )
1025     else:

```

```

1026         raise ValueError(f"Unsupported strategy: {task.strategy}")
1027
1028     _print_run_result(result)
1029     runs.append(result)
1030
1031 ranking_by_margin, ranking_by_auc = _build_rankings(runs)
1032 summary = ExperimentSummary(
1033     preset=preset,
1034     runs=runs,
1035     ranking_by_margin=ranking_by_margin,
1036     ranking_by_auc=ranking_by_auc,
1037 )
1038
1039 csv_path, folds_csv_path, json_path = write_summary_outputs(summary, output_root=output_root)
1040 print(f"\nSaved summary CSV: {csv_path}")
1041 print(f"Saved fold summary CSV: {folds_csv_path}")
1042 print(f"Saved summary JSON: {json_path}")
1043
1044 return summary
1045
1046
1047 def _build_parser() -> argparse.ArgumentParser:
1048     """Build the CLI parser for modular experiment presets."""
1049     parser = argparse.ArgumentParser(
1050         description="Run modular experiment presets for prediction-market models."
1051     )
1052     parser.add_argument(
1053         "--preset",
1054         choices=[*PRESET_ORDER, "all"],
1055         default="all",
1056         help="Preset to run (default: all).",
1057     )
1058     parser.add_argument(
1059         "--mapped-markets-path",
1060         type=Path,
1061         default=Path("datasets/mapped_markets"),
1062         help="Path to mapped market folder (default: datasets/mapped_markets).",
1063     )
1064     parser.add_argument(
1065         "--ratios",
1066         type=str,
1067         default=None,
1068         help="Optional CSV ratios override, e.g. 0.33,0.67,0.95",
1069     )
1070     parser.add_argument(
1071         "--gru-candles",
1072         type=str,
1073         default=None,
1074         help="Optional CSV GRU candle override, e.g. 40,60,80",
1075     )
1076     parser.add_argument(
1077         "--output-root",
1078         type=Path,
1079         default=Path("outputs"),
1080         help="Experiment output root (default: outputs).",
1081     )
1082     parser.add_argument(
1083         "--cv-folds",
1084         type=int,
1085         default=5,
1086         help="Number of stratified CV folds when CV is enabled (default: 5).",
1087     )
1088     parser.add_argument(
1089         "--disable-cv",
1090         action="store_true",
1091         help="Disable cross-validation and use legacy single train/val split.",
1092     )
1093     return parser
1094
1095
1096 def main() -> None:
1097     """Run the experiment runner CLI and dispatch the requested presets."""
1098     args = _build_parser().parse_args()
1099     ratios_override = _parse_csv_floats(args.ratios)

```

```

1100 gru_candles_override = _parse_csv_ints(args.gru_candles)
1101
1102 if args.preset == "all":
1103     for preset in PRESET_ORDER:
1104         print(f"\n===== {preset} =====")
1105         run_preset(
1106             preset,
1107             mapped_markets_path=args.mapped_markets_path,
1108             output_root=args.output_root,
1109             ratios_override=ratios_override,
1110             gru_candles_override=gru_candles_override,
1111             cv_folds=args.cv_folds,
1112             disable_cv=args.disable_cv,
1113         )
1114     else:
1115         run_preset(
1116             args.preset,
1117             mapped_markets_path=args.mapped_markets_path,
1118             output_root=args.output_root,
1119             ratios_override=ratios_override,
1120             gru_candles_override=gru_candles_override,
1121             cv_folds=args.cv_folds,
1122             disable_cv=args.disable_cv,
1123         )
1124
1125
1126 if __name__ == "__main__":
1127     main()
1128
1129
1130
1131 =====
1132 FILE: analysis_and_experiments/plotting/__init__.py
1133 =====
1134 """Plotting helpers."""
1135
1136 from .confusion_matrix import save_confusion_matrix_plot
1137 from .roc import save_roc_plot
1138 from .training_curves import save_train_val_metric_plot
1139
1140 __all__ = [
1141     "save_roc_plot",
1142     "save_confusion_matrix_plot",
1143     "save_train_val_metric_plot",
1144 ]
1145
1146
1147
1148 =====
1149 FILE: analysis_and_experiments/plotting/confusion_matrix.py
1150 =====
1151 import os
1152 from pathlib import Path
1153 from typing import Sequence
1154
1155 os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
1156
1157 import matplotlib
1158
1159 matplotlib.use("Agg")
1160 import matplotlib.pyplot as plt
1161 import numpy as np
1162 from sklearn.metrics import confusion_matrix
1163
1164
1165 def save_confusion_matrix_plot(
1166     *,
1167     output_path: Path,
1168     title: str,
1169     y_true: Sequence[int],
1170     y_pred: Sequence[int],
1171     class_names: tuple[str, str] = ("No", "Yes"),
1172     dpi: int = 300,
1173 ) -> Path:

```

```

1174 """Render and save a confusion matrix image."""
1175 matrix = confusion_matrix(y_true, y_pred, labels=[0, 1])
1176
1177 output_path.parent.mkdir(parents=True, exist_ok=True)
1178
1179 fig, ax = plt.subplots(figsize=(6, 5))
1180 image = ax.imshow(matrix, cmap="Blues")
1181 fig.colorbar(image, ax=ax)
1182
1183 ticks = np.arange(len(class_names))
1184 ax.set_xticks(ticks)
1185 ax.set_yticks(ticks)
1186 ax.set_xticklabels(class_names)
1187 ax.set_yticklabels(class_names)
1188 ax.set_xlabel("Predicted Label")
1189 ax.set_ylabel("True Label")
1190 ax.set_title(title)
1191
1192 threshold = matrix.max() / 2.0 if matrix.size else 0.0
1193 for row_index in range(matrix.shape[0]):
1194     for col_index in range(matrix.shape[1]):
1195         value = int(matrix[row_index, col_index])
1196         text_color = "white" if matrix[row_index, col_index] > threshold else "black"
1197         ax.text(
1198             col_index,
1199             row_index,
1200             f"{value}",
1201             ha="center",
1202             va="center",
1203             color=text_color,
1204         )
1205
1206 fig.tight_layout()
1207 fig.savefig(output_path, dpi=dpi)
1208 plt.close(fig)
1209 return output_path
1210
1211
1212
1213 =====
1214 FILE: analysis_and_experiments/plotting/roc.py
1215 =====
1216 import os
1217 from pathlib import Path
1218
1219 os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
1220
1221 import matplotlib
1222
1223 matplotlib.use("Agg")
1224 import matplotlib.pyplot as plt
1225
1226
1227 def save_roc_plot(
1228     *,
1229     output_path: Path,
1230     title: str,
1231     roc_fpr: list[float] | None,
1232     roc_tpr: list[float] | None,
1233     auc: float | None,
1234     dpi: int = 300,
1235 ) -> Path | None:
1236     """Render and save an ROC curve when ROC data is available."""
1237     if roc_fpr is None or roc_tpr is None or auc is None:
1238         return None
1239
1240     output_path.parent.mkdir(parents=True, exist_ok=True)
1241
1242     plt.figure(figsize=(7, 5))
1243     plt.plot(roc_fpr, roc_tpr, label=f"AUC={auc:.4f}", linewidth=2)
1244     plt.plot([0.0, 1.0], [0.0, 1.0], linestyle="--", linewidth=1)
1245     plt.xlabel("False Positive Rate")
1246     plt.ylabel("True Positive Rate")
1247     plt.title(title)

```

```

1248 plt.legend(loc="lower right")
1249 plt.tight_layout()
1250 plt.savefig(output_path, dpi=dpi)
1251 plt.close()
1252 return output_path
1253
1254
1255
1256 =====
1257 FILE: analysis_and_experiments/plotting/training_curves.py
1258 =====
1259 import os
1260 from pathlib import Path
1261 from typing import Sequence
1262
1263 os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
1264
1265 import matplotlib
1266
1267 matplotlib.use("Agg")
1268 import matplotlib.pyplot as plt
1269
1270
1271 def save_train_val_metric_plot(
1272     *,
1273     output_path: Path,
1274     title: str,
1275     metric_name: str,
1276     train_values: Sequence[float],
1277     val_values: Sequence[float],
1278     y_limit: tuple[float, float] | None = None,
1279     dpi: int = 300,
1280 ) -> Path:
1281     """Plot and save train versus validation metric curves across epochs."""
1282     if len(train_values) != len(val_values):
1283         raise ValueError("train_values and val_values must have the same length.")
1284     if not train_values:
1285         raise ValueError("train_values cannot be empty.")
1286
1287     output_path.parent.mkdir(parents=True, exist_ok=True)
1288
1289     epochs = list(range(1, len(train_values) + 1))
1290     fig, axis = plt.subplots(figsize=(8, 5))
1291     axis.plot(
1292         epochs,
1293         list(train_values),
1294         color="#1f77b4",
1295         linewidth=2,
1296         label=f"Train {metric_name}",
1297     )
1298     axis.plot(
1299         epochs,
1300         list(val_values),
1301         color="#ff7f0e",
1302         linewidth=2,
1303         label=f"Val {metric_name}",
1304     )
1305
1306     axis.set_xlabel("Epoch")
1307     axis.set_ylabel(metric_name)
1308     axis.set_title(title)
1309     if y_limit is not None:
1310         axis.set_ylim(*y_limit)
1311     axis.legend(loc="best")
1312
1313     fig.tight_layout()
1314     fig.savefig(output_path, dpi=dpi)
1315     plt.close(fig)
1316     return output_path
1317
1318
1319
1320 =====
1321 FILE: analysis_and_experiments/strategies/__init__.py

```

```

1322 =====
1323 """Strategy training modules."""
1324
1325 from .common import BinaryMetrics, ExperimentSummary, FoldResult, RunResult, average_binary_metrics
1326 from .gru import run_gru_experiment_on_ratio
1327 from .tabular import run_tabular_experiment_on_ratio
1328
1329 __all__ = [
1330     "BinaryMetrics",
1331     "ExperimentSummary",
1332     "FoldResult",
1333     "RunResult",
1334     "average_binary_metrics",
1335     "run_gru_experiment_on_ratio",
1336     "run_tabular_experiment_on_ratio",
1337 ]
1338
1339
1340
1341 =====
1342 FILE: analysis_and_experiments/strategies/common.py
1343 =====
1344 from dataclasses import asdict, dataclass
1345 from pathlib import Path
1346 from typing import Any
1347
1348
1349 @dataclass
1350 class BinaryMetrics:
1351     sklearn_log_loss: float
1352     torch_bce_loss: float
1353     accuracy: float
1354     precision: float
1355     recall: float
1356     f1: float
1357     auc: float | None
1358     roc_fpr: list[float] | None
1359     roc_tpr: list[float] | None
1360
1361
1362 @dataclass
1363 class FoldResult:
1364     fold_index: int
1365     train_size: int
1366     val_size: int
1367     train_metrics: BinaryMetrics
1368     val_metrics: BinaryMetrics
1369     average_margin_of_victory: float
1370
1371     def to_dict(self) -> dict[str, Any]:
1372         """Serialize the fold result dataclass to a dictionary."""
1373         return asdict(self)
1374
1375
1376 @dataclass
1377 class RunResult:
1378     preset: str
1379     strategy: str
1380     truncate_and_keep_ratio: float
1381     desired_num_candlesticks: int | None
1382     cv_enabled: bool
1383     cv_folds: int | None
1384     train_metrics: BinaryMetrics
1385     val_metrics: BinaryMetrics
1386     average_margin_of_victory: float
1387     fold_results: list[FoldResult]
1388     roc_plot_path: Path | None
1389     gru_train_curve_plot_path: Path | None = None
1390     gru_val_curve_plot_path: Path | None = None
1391     train_confusion_matrix_plot_path: Path | None = None
1392     val_confusion_matrix_plot_path: Path | None = None
1393
1394     def to_dict(self) -> dict[str, Any]:
1395         """Serialize the run result dataclass with path fields converted to strings."""

```

```

1396     payload = asdict(self)
1397     payload["roc_plot_path"] = str(self.roc_plot_path) if self.roc_plot_path is not None else None
1398     payload["gru_train_curve_plot_path"] = (
1399         str(self.gru_train_curve_plot_path)
1400         if self.gru_train_curve_plot_path is not None
1401         else None
1402     )
1403     payload["gru_val_curve_plot_path"] = (
1404         str(self.gru_val_curve_plot_path) if self.gru_val_curve_plot_path is not None else None
1405     )
1406     payload["train_confusion_matrix_plot_path"] = (
1407         str(self.train_confusion_matrix_plot_path)
1408         if self.train_confusion_matrix_plot_path is not None
1409         else None
1410     )
1411     payload["val_confusion_matrix_plot_path"] = (
1412         str(self.val_confusion_matrix_plot_path)
1413         if self.val_confusion_matrix_plot_path is not None
1414         else None
1415     )
1416     payload["fold_results"] = [fold.to_dict() for fold in self.fold_results]
1417     return payload
1418
1419
1420 @dataclass
1421 class ExperimentSummary:
1422     preset: str
1423     runs: list[RunResult]
1424     ranking_by_margin: list[dict[str, Any]]
1425     ranking_by_auc: list[dict[str, Any]]
1426
1427
1428 def average_binary_metrics(metrics: list[BinaryMetrics]) -> BinaryMetrics:
1429     """Average scalar binary metrics across multiple evaluation results."""
1430     if not metrics:
1431         raise ValueError("metrics cannot be empty.")
1432
1433     auc_values = [metric.auc for metric in metrics if metric.auc is not None]
1434
1435     return BinaryMetrics(
1436         sklearn_log_loss=float(sum(metric.sklearn_log_loss for metric in metrics) / len(metrics)),
1437         torch_bce_loss=float(sum(metric.torch_bce_loss for metric in metrics) / len(metrics)),
1438         accuracy=float(sum(metric.accuracy for metric in metrics) / len(metrics)),
1439         precision=float(sum(metric.precision for metric in metrics) / len(metrics)),
1440         recall=float(sum(metric.recall for metric in metrics) / len(metrics)),
1441         f1=float(sum(metric.f1 for metric in metrics) / len(metrics)),
1442         auc=float(sum(auc_values) / len(auc_values)) if auc_values else None,
1443         roc_fpr=None,
1444         roc_tpr=None,
1445     )
1446
1447
1448
1449 =====
1450 FILE: analysis_and_experiments/strategies/gru.py
1451 =====
1452 from copy import deepcopy
1453 from dataclasses import dataclass
1454 from pathlib import Path
1455 from typing import Any, Callable
1456
1457 import torch
1458 import torch.nn as nn
1459 from sklearn.model_selection import StratifiedKFold
1460
1461 from analysis_and_experiments.data import load_filtered_markets_with_trades
1462 from analysis_and_experiments.evaluation import (
1463     calculate_average_margin_of_victory,
1464     evaluate_binary_metrics,
1465 )
1466 from analysis_and_experiments.plotting import (
1467     save_confusion_matrix_plot,
1468     save_train_val_metric_plot,
1469     save_roc_plot,

```

```

1470 )
1471 from analysis_and_experiments.strategies.common import (
1472     FoldResult,
1473     RunResult,
1474     average_binary_metrics,
1475 )
1476
1477 CLASSIFICATION_THRESHOLD = 0.5
1478
1479 @dataclass
1480 class CandleStick:
1481     open_price: float
1482     close_price: float
1483     low_price: float
1484     high_price: float
1485     total_volume: float
1486     weighted_average_price: float
1487
1488
1489 @dataclass
1490 class GRUTrainingHistory:
1491     train_loss: list[float]
1492     train_accuracy: list[float]
1493     val_loss: list[float]
1494     val_accuracy: list[float]
1495
1496
1497 def get_candlestick_data_from_trades(
1498     trades: list[dict[str, Any]],
1499     interval_trade: int,
1500 ) -> list[CandleStick]:
1501     """Aggregate trades into candlestick records over fixed trade intervals."""
1502     candlesticks: list[CandleStick] = []
1503     for start_index in range(0, len(trades), interval_trade):
1504         interval_trades = trades[start_index : start_index + interval_trade]
1505         if not interval_trades:
1506             continue
1507
1508         open_price = float(interval_trades[0]["price"])
1509         close_price = float(interval_trades[-1]["price"])
1510         low_price = min(float(trade["price"]) for trade in interval_trades)
1511         high_price = max(float(trade["price"]) for trade in interval_trades)
1512         total_volume = sum(float(trade["total_usdc"]) for trade in interval_trades)
1513         if total_volume <= 0:
1514             raise ValueError("Total volume must be > 0 to calculate weighted_average_price.")
1515
1516         weighted_average_price = (
1517             sum(float(trade["price"]) * float(trade["total_usdc"]) for trade in interval_trades)
1518             / total_volume
1519         )
1520         candlesticks.append(
1521             CandleStick(
1522                 open_price=open_price,
1523                 close_price=close_price,
1524                 low_price=low_price,
1525                 high_price=high_price,
1526                 total_volume=total_volume,
1527                 weighted_average_price=weighted_average_price,
1528             )
1529         )
1530     )
1531     return candlesticks
1532
1533
1534 def get_expected_candlestick_interval(num_trades: int, desired_num_candlesticks: int) -> int:
1535     """Choose a trade interval that yields roughly the target candlestick count."""
1536     return max(1, num_trades // desired_num_candlesticks)
1537
1538
1539 def build_gru_dataset(
1540     loaded_markets: list[dict[str, Any]],
1541     market_id_to_candlestick_data: dict[str, list[CandleStick]],
1542 ) -> tuple[torch.Tensor, torch.Tensor]:
1543     """Build GRU feature and label tensors from per-market candlestick sequences."""

```

```

1344 X: list[list[list[float]]] = []
1345 y: list[int] = []
1346 for market in loaded_markets:
1347     market_id = str(market["id"])
1348     candlestick_data = market_id_to_candlestick_data[market_id]
1349     market_X = [
1350         [
1351             candlestick.open_price,
1352             candlestick.close_price,
1353             candlestick.low_price,
1354             candlestick.high_price,
1355             candlestick.total_volume,
1356             candlestick.weighted_average_price,
1357         ]
1358         for candlestick in candlestick_data
1359     ]
1360     X.append(market_X)
1361     y.append(1 if str(market.get("outcome", "")).lower() == "yes" else 0)
1362
1363 min_candlestick_length = min(len(market_X) for market_X in X)
1364 X = [market_X[:min_candlestick_length] for market_X in X]
1365 X_tensor = torch.tensor(X, dtype=torch.float32)
1366
1367 # Normalize volume feature market-by-market.
1368 for sample_index in range(X_tensor.shape[0]):
1369     total_volume = X_tensor[sample_index, :, 4]
1370     max_volume = torch.max(total_volume)
1371     min_volume = torch.min(total_volume)
1372     if max_volume == min_volume:
1373         X_tensor[sample_index, :, 4] = 0.0
1374     else:
1375         X_tensor[sample_index, :, 4] = (total_volume - min_volume) / (max_volume - min_volume)
1376
1377 y_tensor = torch.tensor(y, dtype=torch.long).unsqueeze(1)
1378 return X_tensor, y_tensor
1379
1380
1381 def _split_by_indices(
1382     X: torch.Tensor,
1383     y: torch.Tensor,
1384     train_indices: torch.Tensor,
1385     val_indices: torch.Tensor,
1386 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
1387     """Split GRU tensors into train and validation subsets by index."""
1388     X_train = X[train_indices]
1389     y_train = y[train_indices]
1390     X_val = X[val_indices]
1391     y_val = y[val_indices]
1392     return X_train, y_train, X_val, y_val
1393
1394
1395 def prepare_data_for_gru_training(
1396     loaded_markets: list[dict[str, Any]],
1397     market_id_to_candlestick_data: dict[str, list[CandleStick]],
1398     *,
1399     val_ratio: float,
1400     seed: int,
1401 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
1402     """Build, shuffle, and split the GRU dataset for a single validation run."""
1403     X_tensor, y_tensor = build_gru_dataset(
1404         loaded_markets,
1405         market_id_to_candlestick_data,
1406     )
1407
1408     generator = torch.Generator().manual_seed(seed)
1409     shuffled_indices = torch.randperm(X_tensor.shape[0], generator=generator)
1410     X_shuffled = X_tensor[shuffled_indices]
1411     y_shuffled = y_tensor[shuffled_indices]
1412
1413     train_size = int((1 - val_ratio) * X_shuffled.shape[0])
1414     if train_size <= 0 or train_size >= X_shuffled.shape[0]:
1415         raise ValueError(
1416             f"Invalid split with val_ratio={val_ratio} for dataset size={X_shuffled.shape[0]}."
1417         )

```

```

1018 X_train, y_train = X_shuffled[:train_size], y_shuffled[:train_size]
1019 X_val, y_val = X_shuffled[train_size:], y_shuffled[train_size:]
1020
1021 return X_train, y_train, X_val, y_val
1022
1023
1024 def gru_training(
1025     X_train: torch.Tensor,
1026     y_train: torch.Tensor,
1027     X_val: torch.Tensor,
1028     y_val: torch.Tensor,
1029     *,
1030     seed: int,
1031 ) -> tuple[nn.GRU, nn.Linear, GRUTrainingHistory]:
1032     """Train the GRU classifier and return the best checkpoint and history."""
1033     torch.manual_seed(seed)
1034
1035     feature_size = X_train.shape[2]
1036     hidden_size = 32
1037     num_layers = 2
1038     output_size = 1
1039
1040     gru_model = nn.GRU(
1041         input_size=feature_size,
1042         hidden_size=hidden_size,
1043         num_layers=num_layers,
1044         batch_first=True,
1045     )
1046     linear_layer = nn.Linear(hidden_size, output_size)
1047
1048     criterion = nn.BCEWithLogitsLoss()
1049     optimizer = torch.optim.Adam(list(gru_model.parameters()) + list(linear_layer.parameters()), lr=0.001)
1050     num_epochs = 200
1051
1052     best_gru_model_state = deepcopy(gru_model.state_dict())
1053     best_linear_layer_state = deepcopy(linear_layer.state_dict())
1054     best_val_accuracy = 0.0
1055     train_loss_history: list[float] = []
1056     train_accuracy_history: list[float] = []
1057     val_loss_history: list[float] = []
1058     val_accuracy_history: list[float] = []
1059
1060     for epoch in range(num_epochs):
1061         gru_model.train()
1062         optimizer.zero_grad()
1063         output, _ = gru_model(X_train)
1064         last_hidden_state = output[:, -1, :]
1065         logits = linear_layer(last_hidden_state)
1066         loss = criterion(logits, y_train.float())
1067         loss.backward()
1068         optimizer.step()
1069
1070         predicted = (torch.sigmoid(logits) > CLASSIFICATION_THRESHOLD).long()
1071         train_correct = (predicted == y_train).sum().item()
1072         train_total = y_train.size(0)
1073         train_loss = float(loss.item())
1074         train_accuracy = float(train_correct / train_total)
1075
1076         gru_model.eval()
1077         with torch.no_grad():
1078             val_output, _ = gru_model(X_val)
1079             val_last_hidden_state = val_output[:, -1, :]
1080             val_logits = linear_layer(val_last_hidden_state)
1081             val_loss = float(criterion(val_logits, y_val.float()).item())
1082             val_predicted = (torch.sigmoid(val_logits) > CLASSIFICATION_THRESHOLD).long()
1083             val_correct = (val_predicted == y_val).sum().item()
1084             val_total = y_val.size(0)
1085             val_accuracy = val_correct / val_total
1086
1087         train_loss_history.append(train_loss)
1088         train_accuracy_history.append(train_accuracy)
1089         val_loss_history.append(val_loss)
1090         val_accuracy_history.append(float(val_accuracy))
1091

```

```

1692         if val_accuracy > best_val_accuracy:
1693             best_val_accuracy = val_accuracy
1694             best_gru_model_state = deepcopy(gru_model.state_dict())
1695             best_linear_layer_state = deepcopy(linear_layer.state_dict())
1696
1697         if (epoch + 1) % 10 == 0:
1698             print(
1699                 f"Epoch [{epoch + 1}/{num_epochs}], "
1700                 f"Train Loss: {train_loss:.4f}, Train Acc: {train_accuracy:.4f}, "
1701                 f"Val Loss: {val_loss:.4f}, Val Acc: {val_correct / val_total:.4f}"
1702             )
1703
1704     print(f"Loading best model state with Val Accuracy: {best_val_accuracy:.4f}")
1705     gru_model.load_state_dict(best_gru_model_state)
1706     linear_layer.load_state_dict(best_linear_layer_state)
1707     return (
1708         gru_model,
1709         linear_layer,
1710         GRUTrainingHistory(
1711             train_loss=train_loss_history,
1712             train_accuracy=train_accuracy_history,
1713             val_loss=val_loss_history,
1714             val_accuracy=val_accuracy_history,
1715         ),
1716     )
1717
1718
1719 def _predict_probabilities(gru_model: nn.Module, linear_layer: nn.Module, X: torch.Tensor) -> list[float]:
1720     """Predict positive-class probabilities for GRU inputs."""
1721     gru_model.eval()
1722     with torch.no_grad():
1723         output, _ = gru_model(X)
1724         last_hidden_state = output[:, -1, :]
1725         logits = linear_layer(last_hidden_state)
1726         probabilities = torch.sigmoid(logits).squeeze(1).detach().cpu().numpy().tolist()
1727         return [float(value) for value in probabilities]
1728
1729
1730 def _validate_cv_folds(y: torch.Tensor, cv_folds: int) -> None:
1731     """Validate that each class has enough samples for the requested CV folds."""
1732     if cv_folds < 2:
1733         raise ValueError(f"cv_folds must be >= 2 when CV is enabled, got {cv_folds}.")
1734
1735     labels = y.detach().cpu().squeeze(1)
1736     class_counts = torch.bincount(labels, minlength=2)
1737     min_class_count = int(torch.min(class_counts).item())
1738     if min_class_count < cv_folds:
1739         raise ValueError(
1740             "Invalid cv_folds: smallest class count is "
1741             f"{min_class_count}, but cv_folds={cv_folds}. "
1742             "Reduce --cv-folds or disable CV."
1743         )
1744
1745
1746 def _average_histories(histories: list[GRUTrainingHistory]) -> GRUTrainingHistory:
1747     """Average GRU training histories across cross-validation folds."""
1748     if not histories:
1749         raise ValueError("histories cannot be empty.")
1750
1751     num_epochs = len(histories[0].train_loss)
1752     for history in histories:
1753         if len(history.train_loss) != num_epochs:
1754             raise ValueError("All history objects must share the same train_loss length.")
1755         if len(history.train_accuracy) != num_epochs:
1756             raise ValueError("All history objects must share the same train_accuracy length.")
1757         if len(history.val_loss) != num_epochs:
1758             raise ValueError("All history objects must share the same val_loss length.")
1759         if len(history.val_accuracy) != num_epochs:
1760             raise ValueError("All history objects must share the same val_accuracy length.")
1761
1762     return GRUTrainingHistory(
1763         train_loss=[
1764             float(sum(history.train_loss[epoch] for history in histories) / len(histories))
1765             for epoch in range(num_epochs)
1766         ]
1767     )

```

```

1766     ],
1767     train_accuracy=[
1768         float(sum(history.train_accuracy[epoch] for history in histories) / len(histories))
1769         for epoch in range(num_epochs)
1770     ],
1771     val_loss=[
1772         float(sum(history.val_loss[epoch] for history in histories) / len(histories))
1773         for epoch in range(num_epochs)
1774     ],
1775     val_accuracy=[
1776         float(sum(history.val_accuracy[epoch] for history in histories) / len(histories))
1777         for epoch in range(num_epochs)
1778     ],
1779 )
1780
1781
1782 def run_gru_experiment_on_ratio(
1783     mapped_market_folder_path: Path,
1784     truncate_and_keep_ratio: float,
1785     *,
1786     preset: str,
1787     desired_num_candlesticks: int = 40,
1788     market_policy: Callable[[dict[str, Any]], bool] | None = None,
1789     val_ratio: float = 0.2,
1790     seed: int = 42,
1791     use_cross_validation: bool = True,
1792     cv_folds: int = 5,
1793     roc_output_dir: Path,
1794     train_val_curves_output_dir: Path | None = None,
1795     confusion_matrix_output_dir: Path | None = None,
1796 ) -> RunResult:
1797     """Run the GRU experiment for one truncation ratio and save its artifacts."""
1798     if train_val_curves_output_dir is None:
1799         train_val_curves_output_dir = roc_output_dir.parent.parent / "training_curves" / "gru"
1800     if confusion_matrix_output_dir is None:
1801         confusion_matrix_output_dir = roc_output_dir.parent.parent / "confusion_matrix" / "gru"
1802
1803     if market_policy is None:
1804         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
1805             mapped_market_folder_path,
1806             truncate_and_keep_ratio,
1807         )
1808     else:
1809         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
1810             mapped_market_folder_path,
1811             truncate_and_keep_ratio,
1812             market_policy=market_policy,
1813         )
1814     print(
1815         f"Loaded {len(loaded_markets)} markets after filtering from {total_market_folders} market folders."
1816     )
1817
1818     loaded_market_id_to_candlestick_data: dict[str, list[CandleStick]] = {}
1819     for market in loaded_markets:
1820         market_id = str(market["id"])
1821         trades = loaded_market_id_to_trades[market_id]
1822         interval_trade = get_expected_candlestick_interval(len(trades), desired_num_candlesticks)
1823         loaded_market_id_to_candlestick_data[market_id] = get_candlestick_data_from_trades(
1824             trades,
1825             interval_trade,
1826         )
1827
1828     X, y = build_gru_dataset(
1829         loaded_markets,
1830         loaded_market_id_to_candlestick_data,
1831     )
1832
1833     fold_results: list[FoldResult] = []
1834     y_true_train: list[int]
1835     y_prob_train: list[float]
1836     y_true_val: list[int]
1837     y_prob_val: list[float]
1838     average_margin_of_victory: float
1839     training_history: GRUTrainingHistory

```

```

1840 train_metrics: Any
1841 val_metrics: Any
1842
1843 if use_cross_validation:
1844     _validate_cv_folds(y, cv_folds)
1845     y_labels = y.detach().cpu().squeeze(1).numpy()
1846     splitter = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=seed)
1847
1848     fold_histories: list[GRUTrainingHistory] = []
1849     train_metrics_per_fold = []
1850     all_train_true: list[int] = []
1851     all_train_prob: list[float] = []
1852     all_val_true: list[int] = []
1853     all_val_prob: list[float] = []
1854     all_val_reference_prices: list[float] = []
1855
1856     for fold_index, (train_idx_np, val_idx_np) in enumerate(
1857         splitter.split(X.detach().cpu().numpy(), y_labels),
1858         start=1,
1859     ):
1860         train_indices = torch.tensor(train_idx_np, dtype=torch.long)
1861         val_indices = torch.tensor(val_idx_np, dtype=torch.long)
1862         X_train, y_train_tensor, X_val, y_val_tensor = _split_by_indices(
1863             X,
1864             y,
1865             train_indices,
1866             val_indices,
1867         )
1868
1869         print(
1870             f"Prepared GRU CV fold {fold_index}/{cv_folds} for ratio={truncate_and_keep_ratio:.2f}, "
1871             f"candles={desired_num_candlesticks}. X_train={tuple(X_train.shape)}, X_val={tuple(X_val.shape)}"
1872         )
1873
1874         gru_model, linear_layer, fold_history = gru_training(
1875             X_train,
1876             y_train_tensor,
1877             X_val,
1878             y_val_tensor,
1879             seed=seed,
1880         )
1881         fold_histories.append(fold_history)
1882
1883         fold_prob_train = _predict_probabilities(gru_model, linear_layer, X_train)
1884         fold_prob_val = _predict_probabilities(gru_model, linear_layer, X_val)
1885
1886         fold_train_metrics = evaluate_binary_metrics(
1887             y_train_tensor,
1888             fold_prob_train,
1889             classification_threshold=CLASSIFICATION_THRESHOLD,
1890         )
1891         fold_val_metrics = evaluate_binary_metrics(
1892             y_val_tensor,
1893             fold_prob_val,
1894             classification_threshold=CLASSIFICATION_THRESHOLD,
1895         )
1896         train_metrics_per_fold.append(fold_train_metrics)
1897
1898         fold_reference_prices = [float(value) for value in X_val[:, -1, 5].detach().cpu().tolist()]
1899         fold_margin = calculate_average_margin_of_victory(fold_prob_val, fold_reference_prices)
1900
1901         fold_results.append(
1902             FoldResult(
1903                 fold_index=fold_index,
1904                 train_size=int(y_train_tensor.shape[0]),
1905                 val_size=int(y_val_tensor.shape[0]),
1906                 train_metrics=fold_train_metrics,
1907                 val_metrics=fold_val_metrics,
1908                 average_margin_of_victory=fold_margin,
1909             )
1910         )
1911
1912     all_train_true.extend(
1913         int(value) for value in y_train_tensor.detach().cpu().squeeze(1).tolist()

```

```

1914         )
1915         all_train_prob.extend(fold_prob_train)
1916         all_val_true.extend(int(value) for value in y_val_tensor.detach().cpu().squeeze(1).tolist())
1917         all_val_prob.extend(fold_prob_val)
1918         all_val_reference_prices.extend(fold_reference_prices)
1919
1920     print(
1921         f"GRU CV fold {fold_index}/{cv_folds} completed. "
1922         f"train={y_train_tensor.shape[0]}, val={y_val_tensor.shape[0]}"
1923     )
1924
1925     y_true_train = all_train_true
1926     y_prob_train = all_train_prob
1927     y_true_val = all_val_true
1928     y_prob_val = all_val_prob
1929     average_margin_of_victory = calculate_average_margin_of_victory(
1930         all_val_prob,
1931         all_val_reference_prices,
1932     )
1933
1934     training_history = _average_histories(fold_histories)
1935     train_metrics = average_binary_metrics(train_metrics_per_fold)
1936     val_metrics = evaluate_binary_metrics(
1937         torch.tensor(all_val_true, dtype=torch.long).unsqueeze(1),
1938         all_val_prob,
1939         classification_threshold=CLASSIFICATION_THRESHOLD,
1940     )
1941     file_suffix = f"_cv{cv_folds}"
1942     cv_enabled = True
1943     run_cv_folds: int | None = cv_folds
1944 else:
1945     X_train, y_train_tensor, X_val, y_val_tensor = prepare_data_for_gru_training(
1946         loaded_markets,
1947         loaded_market_id_to_candlestick_data,
1948         val_ratio=val_ratio,
1949         seed=seed,
1950     )
1951     print(
1952         f"Prepared GRU data for ratio={truncate_and_keep_ratio:.2f}, candles={desired_num_candlesticks}. "
1953         f"X_train={tuple(X_train.shape)}, X_val={tuple(X_val.shape)}"
1954     )
1955
1956     gru_model, linear_layer, training_history = gru_training(
1957         X_train,
1958         y_train_tensor,
1959         X_val,
1960         y_val_tensor,
1961         seed=seed,
1962     )
1963
1964     y_prob_train = _predict_probabilities(gru_model, linear_layer, X_train)
1965     y_prob_val = _predict_probabilities(gru_model, linear_layer, X_val)
1966     y_true_train = [int(value) for value in y_train_tensor.detach().cpu().squeeze(1).tolist()]
1967     y_true_val = [int(value) for value in y_val_tensor.detach().cpu().squeeze(1).tolist()]
1968
1969     train_metrics = evaluate_binary_metrics(
1970         y_train_tensor,
1971         y_prob_train,
1972         classification_threshold=CLASSIFICATION_THRESHOLD,
1973     )
1974     val_metrics = evaluate_binary_metrics(
1975         y_val_tensor,
1976         y_prob_val,
1977         classification_threshold=CLASSIFICATION_THRESHOLD,
1978     )
1979
1980     reference_prices = [float(value) for value in X_val[:, -1, 5].detach().cpu().tolist()]
1981     average_margin_of_victory = calculate_average_margin_of_victory(y_prob_val, reference_prices)
1982     fold_results = [
1983         FoldResult(
1984             fold_index=1,
1985             train_size=int(y_train_tensor.shape[0]),
1986             val_size=int(y_val_tensor.shape[0]),
1987             train_metrics=train_metrics,

```

```

1988         val_metrics=val_metrics,
1989         average_margin_of_victory=average_margin_of_victory,
1990     )
1991 ]
1992 file_suffix = ""
1993 cv_enabled = False
1994 run_cv_folds = None
1995
1996 roc_file_name = (
1997     f"{preset}_ratio_{truncate_and_keep_ratio:.2f}_candles_{desired_num_candlesticks}{file_suffix}.png"
1998 )
1999 roc_plot_path = save_roc_plot(
2000     output_path=roc_output_dir / roc_file_name,
2001     title=(
2002         f"GRU ROC (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2003         f"candles={desired_num_candlesticks}, mode={'cv' if use_cross_validation else 'single'})"
2004     ),
2005     roc_fpr=val_metrics.roc_fpr,
2006     roc_tpr=val_metrics.roc_tpr,
2007     auc=val_metrics.auc,
2008     dpi=300,
2009 )
2010
2011 curve_suffix = file_suffix
2012 loss_curve_file_name = (
2013     f"{preset}_ratio_{truncate_and_keep_ratio:.2f}_candles_{desired_num_candlesticks}{curve_suffix}_loss.png"
2014 )
2015 accuracy_curve_file_name = (
2016     f"{preset}_ratio_{truncate_and_keep_ratio:.2f}_candles_{desired_num_candlesticks}{curve_suffix}_accuracy.png"
2017 )
2018 loss_curve_plot_path = save_train_val_metric_plot(
2019     output_path=train_val_curves_output_dir / loss_curve_file_name,
2020     title=(
2021         f"GRU Train/Val Loss (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2022         f"candles={desired_num_candlesticks}, mode={'cv' if use_cross_validation else 'single'})"
2023     ),
2024     metric_name="Loss",
2025     train_values=training_history.train_loss,
2026     val_values=training_history.val_loss,
2027     dpi=300,
2028 )
2029 accuracy_curve_plot_path = save_train_val_metric_plot(
2030     output_path=train_val_curves_output_dir / accuracy_curve_file_name,
2031     title=(
2032         f"GRU Train/Val Accuracy (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2033         f"candles={desired_num_candlesticks}, mode={'cv' if use_cross_validation else 'single'})"
2034     ),
2035     metric_name="Accuracy",
2036     train_values=training_history.train_accuracy,
2037     val_values=training_history.val_accuracy,
2038     y_limit=(0.0, 1.0),
2039     dpi=300,
2040 )
2041
2042 y_pred_train = [1 if probability >= CLASSIFICATION_THRESHOLD else 0 for probability in y_prob_train]
2043 y_pred_val = [1 if probability >= CLASSIFICATION_THRESHOLD else 0 for probability in y_prob_val]
2044
2045 train_confusion_matrix_file_name = (
2046     f"{preset}_ratio_{truncate_and_keep_ratio:.2f}_candles_{desired_num_candlesticks}{file_suffix}_train.png"
2047 )
2048 val_confusion_matrix_file_name = (
2049     f"{preset}_ratio_{truncate_and_keep_ratio:.2f}_candles_{desired_num_candlesticks}{file_suffix}_val.png"
2050 )
2051 train_confusion_matrix_plot_path = save_confusion_matrix_plot(
2052     output_path=confusion_matrix_output_dir / train_confusion_matrix_file_name,
2053     title=(
2054         f"GRU Train Confusion Matrix (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2055         f"candles={desired_num_candlesticks}, mode={'cv' if use_cross_validation else 'single'})"
2056     ),
2057     y_true=y_true_train,
2058     y_pred=y_pred_train,
2059     dpi=300,
2060 )
2061 val_confusion_matrix_plot_path = save_confusion_matrix_plot(

```

```

2062     output_path=confusion_matrix_output_dir / val_confusion_matrix_file_name,
2063     title=(
2064         f"GRU Val Confusion Matrix (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2065         f"candles={desired_num_candlesticks}, mode={'cv' if use_cross_validation else 'single'})"
2066     ),
2067     y_true=y_true_val,
2068     y_pred=y_pred_val,
2069     dpi=300,
2070 )
2071
2072     return RunResult(
2073         preset=preset,
2074         strategy="gru",
2075         truncate_and_keep_ratio=truncate_and_keep_ratio,
2076         desired_num_candlesticks=desired_num_candlesticks,
2077         cv_enabled=cv_enabled,
2078         cv_folds=run_cv_folds,
2079         train_metrics=train_metrics,
2080         val_metrics=val_metrics,
2081         average_margin_of_victory=average_margin_of_victory,
2082         fold_results=fold_results,
2083         roc_plot_path=roc_plot_path,
2084         gru_train_curve_plot_path=loss_curve_plot_path,
2085         gru_val_curve_plot_path=accuracy_curve_plot_path,
2086         train_confusion_matrix_plot_path=train_confusion_matrix_plot_path,
2087         val_confusion_matrix_plot_path=val_confusion_matrix_plot_path,
2088     )
2089
2090
2091
2092     =====
2093     FILE: analysis_and_experiments/strategies/tabular.py
2094     =====
2095     from datetime import datetime
2096     from pathlib import Path
2097     from typing import Any, Callable
2098
2099     import torch
2100     from sklearn.model_selection import StratifiedKFold
2101     from xgboost import XGBClassifier
2102
2103     from analysis_and_experiments.data import load_filtered_markets_with_trades
2104     from analysis_and_experiments.evaluation import (
2105         calculate_average_margin_of_victory,
2106         evaluate_binary_metrics,
2107     )
2108     from analysis_and_experiments.plotting import save_confusion_matrix_plot, save_roc_plot
2109     from analysis_and_experiments.strategies.common import (
2110         FoldResult,
2111         RunResult,
2112         average_binary_metrics,
2113     )
2114
2115     CLASSIFICATION_THRESHOLD = 0.5
2116     REFERENCE_PRICE_FEATURE = "rolling_vwap_last"
2117
2118     XGB_PARAMS: dict[str, Any] = {
2119         "objective": "binary:logistic",
2120         "eval_metric": "logloss",
2121         "random_state": 42,
2122         "n_estimators": 300,
2123         "learning_rate": 0.05,
2124         "max_depth": 4,
2125         "subsample": 0.9,
2126         "colsample_bytree": 0.9,
2127     }
2128
2129
2130     def _safe_mean(values: list[float]) -> float:
2131         """Return the mean of 'values' or '0.0' when the list is empty."""
2132         if not values:
2133             return 0.0
2134         return float(sum(values) / len(values))
2135

```

```

236 def _safe_variance(values: list[float]) -> float:
237     """Return the population variance of "values" or "0.0" for short inputs."""
238     if len(values) < 2:
239         return 0.0
240     mean = _safe_mean(values)
241     return float(sum((value - mean) ** 2 for value in values) / len(values))
242
243
244 def _safe_std(values: list[float]) -> float:
245     """Return the standard deviation derived from "_safe_variance"."""
246     return float(_safe_variance(values) ** 0.5)
247
248
249 def _linear_slope(values: list[float]) -> float:
250     """Estimate the least-squares slope of a numeric sequence."""
251     if len(values) < 2:
252         return 0.0
253
254     x_values = list(range(len(values)))
255     x_mean = _safe_mean(x_values)
256     y_mean = _safe_mean(values)
257     numerator = sum((x - x_mean) * (y - y_mean) for x, y in zip(x_values, values, strict=True))
258     denominator = sum((x - x_mean) ** 2 for x in x_values)
259     if denominator == 0:
260         return 0.0
261     return float(numerator / denominator)
262
263
264 def _parse_trade_arrays(
265     trades: list[dict[str, Any]],
266 ) -> tuple[list[float], list[float], list[float], list[str], list[str]]:
267     """Extract aligned trade feature arrays from raw trade dictionaries."""
268     prices = [float(trade["price"]) for trade in trades]
269     volumes = [float(trade["total_usdc"]) for trade in trades]
270     tokens = [float(trade["tokens"]) for trade in trades]
271     sides = [str(trade["side"]) for trade in trades]
272     wallets = [str(trade["wallet"]) for trade in trades]
273     return prices, volumes, tokens, sides, wallets
274
275
276 def _calculate_duration_seconds(trades: list[dict[str, Any]]) -> float:
277     """Return elapsed seconds between the first and last trade timestamps."""
278     if len(trades) < 2:
279         return 0.0
280     first_ts = datetime.fromisoformat(str(trades[0]["timestamp"]))
281     last_ts = datetime.fromisoformat(str(trades[-1]["timestamp"]))
282     duration = (last_ts - first_ts).total_seconds()
283     return float(max(duration, 0.0))
284
285
286 def _rolling_volume_weighted_prices(prices: list[float], volumes: list[float], window: int) -> list[float]:
287     """Compute rolling volume-weighted prices over a fixed window."""
288     if window < 1:
289         raise ValueError("window must be >= 1")
290
291     rolling_values: list[float] = []
292     for end_index in range(len(prices)):
293         start_index = max(0, end_index - window + 1)
294         window_prices = prices[start_index : end_index + 1]
295         window_volumes = volumes[start_index : end_index + 1]
296
297         volume_sum = float(sum(window_volumes))
298         if volume_sum > 0:
299             weighted_price = sum(
300                 price * volume
301                 for price, volume in zip(window_prices, window_volumes, strict=True)
302             ) / volume_sum
303         else:
304             weighted_price = _safe_mean(window_prices)
305         rolling_values.append(float(weighted_price))
306
307     return rolling_values
308
309

```

```

2210 def extract_market_features(
2211     market: dict[str, Any],
2212     trades: list[dict[str, Any]],
2213     *,
2214     vwap_window: int = 4,
2215     whale_top_k: int = 10,
2216 ) -> dict[str, float]:
2217     """Extract tabular features describing one market's trade history."""
2218     _ = market
2219     if vwap_window < 1:
2220         raise ValueError("vwap_window must be >= 1")
2221     if whale_top_k < 1:
2222         raise ValueError("whale_top_k must be >= 1")
2223     if not trades:
2224         return {}
2225
2226     prices, volumes, tokens, sides, wallets = _parse_trade_arrays(trades)
2227     rolling_weighted_prices = _rolling_volume_weighted_prices(prices, volumes, vwap_window)
2228
2229     duration_seconds = _calculate_duration_seconds(trades)
2230     trades_per_minute = float(len(trades) / (duration_seconds / 60.0)) if duration_seconds > 0 else 0.0
2231
2232     wallet_to_net_tokens: dict[str, float] = {}
2233     for wallet, side, token_count in zip(wallets, sides, tokens, strict=True):
2234         direction = 1.0 if side == "BUY" else -1.0
2235         wallet_to_net_tokens[wallet] = wallet_to_net_tokens.get(wallet, 0.0) + direction * token_count
2236
2237     positive_net_holdings = [value for value in wallet_to_net_tokens.values() if value > 0]
2238     positive_net_holdings.sort(reverse=True)
2239     total_positive_holdings = float(sum(positive_net_holdings))
2240
2241     if total_positive_holdings > 0:
2242         top1_share = float(positive_net_holdings[0] / total_positive_holdings)
2243         topk_share = float(sum(positive_net_holdings[:whale_top_k]) / total_positive_holdings)
2244         whale_hhi = float(sum((value / total_positive_holdings) ** 2 for value in positive_net_holdings))
2245     else:
2246         top1_share = 0.0
2247         topk_share = 0.0
2248         whale_hhi = 0.0
2249
2250     features = {
2251         "price_latest": float(prices[-1]),
2252         "price_min": float(min(prices)),
2253         "price_max": float(max(prices)),
2254         "price_range": float(max(prices) - min(prices)),
2255         "price_mean": _safe_mean(prices),
2256         "price_std": _safe_std(prices),
2257         "price_var": _safe_variance(prices),
2258         "volume_total_usdc": float(sum(volumes)),
2259         "volume_latest_usdc": float(volumes[-1]),
2260         "volume_mean_usdc": _safe_mean(volumes),
2261         "volume_std_usdc": _safe_std(volumes),
2262         "volume_var_usdc": _safe_variance(volumes),
2263         "volume_max_usdc": float(max(volumes)),
2264         "trade_count": float(len(trades)),
2265         "duration_seconds": duration_seconds,
2266         "trades_per_minute": trades_per_minute,
2267         "rolling_vwap_last": float(rolling_weighted_prices[-1]),
2268         "rolling_vwap_mean": _safe_mean(rolling_weighted_prices),
2269         "rolling_vwap_std": _safe_std(rolling_weighted_prices),
2270         "rolling_vwap_min": float(min(rolling_weighted_prices)),
2271         "rolling_vwap_max": float(max(rolling_weighted_prices)),
2272         "rolling_vwap_slope": _linear_slope(rolling_weighted_prices),
2273         "whale_top1_share": top1_share,
2274         "whale_topk_share": topk_share,
2275         "whale_hhi": whale_hhi,
2276     }
2277     return features
2278
2279
2280
2281 def build_tabular_feature_matrix(
2282     loaded_markets: list[dict[str, Any]],
2283     market_id_to_trades: dict[str, list[dict[str, Any]]],

```

```

2384 *,
2385 vwap_window: int = 4,
2386 whale_top_k: int = 10,
2387 ) -> tuple[torch.Tensor, torch.Tensor, list[str], list[str]]:
2388     """Build tabular feature and label tensors for filtered markets."""
2389     rows: list[list[float]] = []
2390     labels: list[int] = []
2391     kept_market_ids: list[str] = []
2392     feature_names: list[str] = []
2393
2394     for market in loaded_markets:
2395         market_id = str(market["id"])
2396         trades = market_id_to_trades.get(market_id, [])
2397         feature_map = extract_market_features(
2398             market,
2399             trades,
2400             vwap_window=vwap_window,
2401             whale_top_k=whale_top_k,
2402         )
2403
2404         if not feature_map:
2405             continue
2406
2407         if not feature_names:
2408             feature_names = list(feature_map.keys())
2409         elif feature_names != list(feature_map.keys()):
2410             raise ValueError("Feature key order mismatch across markets.")
2411
2412         rows.append([float(feature_map[name]) for name in feature_names])
2413         labels.append(1 if str(market["outcome"]).lower() == "yes" else 0)
2414         kept_market_ids.append(market_id)
2415
2416     if not rows:
2417         raise ValueError("No valid market rows were generated for tabular features.")
2418
2419     X = torch.tensor(rows, dtype=torch.float32)
2420     y = torch.tensor(labels, dtype=torch.long).unsqueeze(1)
2421     return X, y, feature_names, kept_market_ids
2422
2423
2424 def _split_and_scale_by_indices(
2425     X: torch.Tensor,
2426     y: torch.Tensor,
2427     train_indices: torch.Tensor,
2428     val_indices: torch.Tensor,
2429 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor, dict[str, torch.Tensor]]:
2430     """Split tabular tensors by index and standardize features from the train split."""
2431     X_train_raw = X[train_indices]
2432     y_train = y[train_indices]
2433     X_val_raw = X[val_indices]
2434     y_val = y[val_indices]
2435
2436     mean = X_train_raw.mean(dim=0)
2437     std = X_train_raw.std(dim=0, unbiased=False)
2438     safe_std = torch.where(std > 0, std, torch.ones_like(std))
2439
2440     X_train = (X_train_raw - mean) / safe_std
2441     X_val = (X_val_raw - mean) / safe_std
2442
2443     scaler_stats = {
2444         "mean": mean,
2445         "std": safe_std,
2446         "train_indices": train_indices,
2447         "val_indices": val_indices,
2448         "X_train_raw": X_train_raw,
2449         "X_val_raw": X_val_raw,
2450     }
2451     return X_train, y_train, X_val, y_val, scaler_stats
2452
2453
2454 def split_and_scale_tabular_dataset(
2455     X: torch.Tensor,
2456     y: torch.Tensor,
2457     *,

```

```

2358     val_ratio: float,
2359     seed: int,
2360 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor, dict[str, torch.Tensor]]:
2361     """Shuffle, split, and standardize a tabular dataset for one validation split."""
2362     if X.ndim != 2:
2363         raise ValueError("X must have shape (num_markets, num_features).")
2364     if y.ndim != 2 or y.shape[1] != 1:
2365         raise ValueError("y must have shape (num_markets, 1).")
2366     if X.shape[0] != y.shape[0]:
2367         raise ValueError("X and y must have the same first dimension.")
2368     if not (0.0 < val_ratio < 1.0):
2369         raise ValueError("val_ratio must be in (0, 1).")
2370     if X.shape[0] < 2:
2371         raise ValueError("Need at least 2 samples to split into train/val.")
2372
2373     num_samples = X.shape[0]
2374     generator = torch.Generator().manual_seed(seed)
2375     permutation = torch.randperm(num_samples, generator=generator)
2376
2377     train_size = int((1.0 - val_ratio) * num_samples)
2378     train_size = max(1, min(train_size, num_samples - 1))
2379
2380     train_indices = permutation[:train_size]
2381     val_indices = permutation[train_size:]
2382
2383     return _split_and_scale_by_indices(X, y, train_indices, val_indices)
2384
2385
2386 def prepare_tabular_dataset(
2387     mapped_market_folder_path: Path,
2388     *,
2389     truncate_and_keep_ratio: float,
2390     market_policy: Callable[[dict[str, Any]], bool] | None = None,
2391     val_ratio: float,
2392     vwap_window: int,
2393     whale_top_k: int,
2394     seed: int,
2395 ) -> tuple[
2396     torch.Tensor,
2397     torch.Tensor,
2398     torch.Tensor,
2399     torch.Tensor,
2400     list[str],
2401     list[str],
2402     dict[str, torch.Tensor],
2403 ]:
2404     """Load mapped markets and prepare a single-split tabular dataset."""
2405     if market_policy is None:
2406         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
2407             mapped_market_folder_path,
2408             truncate_and_keep_ratio,
2409         )
2410     else:
2411         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
2412             mapped_market_folder_path,
2413             truncate_and_keep_ratio,
2414             market_policy=market_policy,
2415         )
2416     print(
2417         f"Loaded {len(loaded_markets)} markets after filtering from {total_market_folders} market folders."
2418     )
2419
2420     X, y, feature_names, kept_market_ids = build_tabular_feature_matrix(
2421         loaded_markets,
2422         loaded_market_id_to_trades,
2423         vwap_window=vwap_window,
2424         whale_top_k=whale_top_k,
2425     )
2426
2427     X_train, y_train, X_val, y_val, scaler_stats = split_and_scale_tabular_dataset(
2428         X,
2429         y,
2430         val_ratio=val_ratio,
2431         seed=seed,

```

```

2432 )
2433 return X_train, y_train, X_val, y_val, feature_names, kept_market_ids, scaler_stats
2434
2435
2436 def _validate_cv_folds(y: torch.Tensor, cv_folds: int) -> None:
2437     """Validate that each class has enough samples for the requested CV folds."""
2438     if cv_folds < 2:
2439         raise ValueError(f"cv_folds must be >= 2 when CV is enabled, got {cv_folds}.")
2440
2441     labels = y.detach().cpu().squeeze(1)
2442     class_counts = torch.bincount(labels, minlength=2)
2443     min_class_count = int(torch.min(class_counts).item())
2444     if min_class_count < cv_folds:
2445         raise ValueError(
2446             "Invalid cv_folds: smallest class count is "
2447             f"{min_class_count}, but cv_folds={cv_folds}. "
2448             "Reduce --cv-folds or disable CV."
2449         )
2450
2451
2452 def _load_tabular_dataset(
2453     mapped_market_folder_path: Path,
2454     *,
2455     truncate_and_keep_ratio: float,
2456     market_policy: Callable[[dict[str, Any]], bool] | None,
2457     vwap_window: int,
2458     whale_top_k: int,
2459 ) -> tuple[torch.Tensor, torch.Tensor, list[str]]:
2460     """Load markets and build the full tabular feature matrix without splitting."""
2461     if market_policy is None:
2462         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
2463             mapped_market_folder_path,
2464             truncate_and_keep_ratio,
2465         )
2466     else:
2467         loaded_markets, loaded_market_id_to_trades, total_market_folders = load_filtered_markets_with_trades(
2468             mapped_market_folder_path,
2469             truncate_and_keep_ratio,
2470             market_policy=market_policy,
2471         )
2472
2473     print(
2474         f"Loaded {len(loaded_markets)} markets after filtering from {total_market_folders} market folders."
2475     )
2476
2477     X, y, feature_names, _kept_market_ids = build_tabular_feature_matrix(
2478         loaded_markets,
2479         loaded_market_id_to_trades,
2480         vwap_window=vwap_window,
2481         whale_top_k=whale_top_k,
2482     )
2483     return X, y, feature_names
2484
2485
2486 def _fit_tabular_model(X_train: torch.Tensor, y_train: torch.Tensor) -> XGBClassifier:
2487     """Fit the XGBoost tabular classifier on training tensors."""
2488     model = XGBClassifier(**XGB_PARAMS)
2489     model.fit(X_train.detach().cpu().numpy(), y_train.detach().cpu().squeeze(1).numpy())
2490     return model
2491
2492
2493 def _predict_tabular_probabilities(model: XGBClassifier, X: torch.Tensor) -> list[float]:
2494     """Predict positive-class probabilities for tabular inputs."""
2495     return [float(value) for value in model.predict_proba(X.detach().cpu().numpy())[:, 1].tolist()]
2496
2497
2498 def _run_tabular_single_split(
2499     *,
2500     X: torch.Tensor,
2501     y: torch.Tensor,
2502     feature_names: list[str],
2503     val_ratio: float,
2504     seed: int,
2505 ) -> tuple[

```

```

2306     list[FoldResult],
2307     list[int],
2308     list[float],
2309     list[float],
2310     torch.Tensor,
2311     float,
2312 ]:
2313     """Train and evaluate the tabular model on one shuffled train/validation split."""
2314     X_train, y_train, X_val, y_val, scaler_stats = split_and_scale_tabular_dataset(
2315         X,
2316         y,
2317         val_ratio=val_ratio,
2318         seed=seed,
2319     )
2320
2321     print(
2322         f"Prepared tabular data with legacy split. "
2323         f"X_train={tuple(X_train.shape)}, X_val={tuple(X_val.shape)}, features={len(feature_names)}"
2324     )
2325
2326     model = _fit_tabular_model(X_train, y_train)
2327     y_prob_train = _predict_tabular_probabilities(model, X_train)
2328     y_prob_val = _predict_tabular_probabilities(model, X_val)
2329
2330     train_metrics = evaluate_binary_metrics(
2331         y_train,
2332         y_prob_train,
2333         classification_threshold=CLASSIFICATION_THRESHOLD,
2334     )
2335     val_metrics = evaluate_binary_metrics(
2336         y_val,
2337         y_prob_val,
2338         classification_threshold=CLASSIFICATION_THRESHOLD,
2339     )
2340
2341     ref_index = feature_names.index(REFERENCE_PRICE_FEATURE)
2342     X_val_raw = scaler_stats["X_val_raw"]
2343     reference_prices = [float(value) for value in X_val_raw[:, ref_index].detach().cpu().tolist()]
2344     margin = calculate_average_margin_of_victory(y_prob_val, reference_prices)
2345
2346     fold_result = FoldResult(
2347         fold_index=1,
2348         train_size=int(y_train.shape[0]),
2349         val_size=int(y_val.shape[0]),
2350         train_metrics=train_metrics,
2351         val_metrics=val_metrics,
2352         average_margin_of_victory=margin,
2353     )
2354
2355     return (
2356         [fold_result],
2357         [int(value) for value in y_train.detach().cpu().squeeze(1).tolist()],
2358         y_prob_train,
2359         y_prob_val,
2360         y_val,
2361         margin,
2362     )
2363
2364
2365 def _run_tabular_cross_validation(
2366     *,
2367     X: torch.Tensor,
2368     y: torch.Tensor,
2369     feature_names: list[str],
2370     seed: int,
2371     cv_folds: int,
2372 ) -> tuple[
2373     list[FoldResult],
2374     list[int],
2375     list[float],
2376     list[float],
2377     float,
2378     torch.Tensor,
2379 ]:

```

```

2380 """Train and evaluate the tabular model with stratified cross-validation."""
2381 _validate_cv_folds(y, cv_folds)
2382
2383 y_labels = y.detach().cpu().squeeze(1).numpy()
2384 splitter = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=seed)
2385
2386 ref_index = feature_names.index(REFERENCE_PRICE_FEATURE)
2387
2388 fold_results: list[FoldResult] = []
2389 all_train_true: list[int] = []
2390 all_train_prob: list[float] = []
2391
2392 oof_val_true: list[int] = []
2393 oof_val_prob: list[float] = []
2394 oof_reference_prices: list[float] = []
2395
2396 for fold_index, (train_idx_np, val_idx_np) in enumerate(
2397     splitter.split(X.detach().cpu().numpy(), y_labels),
2398     start=1,
2399 ):
2400     train_indices = torch.tensor(train_idx_np, dtype=torch.long)
2401     val_indices = torch.tensor(val_idx_np, dtype=torch.long)
2402
2403     X_train, y_train, X_val, y_val, scaler_stats = _split_and_scale_by_indices(
2404         X,
2405         y,
2406         train_indices,
2407         val_indices,
2408     )
2409
2410     model = _fit_tabular_model(X_train, y_train)
2411     y_prob_train = _predict_tabular_probabilities(model, X_train)
2412     y_prob_val = _predict_tabular_probabilities(model, X_val)
2413
2414     train_metrics = evaluate_binary_metrics(
2415         y_train,
2416         y_prob_train,
2417         classification_threshold=CLASSIFICATION_THRESHOLD,
2418     )
2419     val_metrics = evaluate_binary_metrics(
2420         y_val,
2421         y_prob_val,
2422         classification_threshold=CLASSIFICATION_THRESHOLD,
2423     )
2424
2425     X_val_raw = scaler_stats["X_val_raw"]
2426     reference_prices = [float(value) for value in X_val_raw[:, ref_index].detach().cpu().tolist()]
2427     fold_margin = calculate_average_margin_of_victory(y_prob_val, reference_prices)
2428
2429     fold_results.append(
2430         FoldResult(
2431             fold_index=fold_index,
2432             train_size=int(y_train.shape[0]),
2433             val_size=int(y_val.shape[0]),
2434             train_metrics=train_metrics,
2435             val_metrics=val_metrics,
2436             average_margin_of_victory=fold_margin,
2437         )
2438     )
2439     all_train_true.extend(int(value) for value in y_train.detach().cpu().squeeze(1).tolist())
2440     all_train_prob.extend(y_prob_train)
2441
2442     oof_val_true.extend(int(value) for value in y_val.detach().cpu().squeeze(1).tolist())
2443     oof_val_prob.extend(y_prob_val)
2444     oof_reference_prices.extend(reference_prices)
2445
2446     print(
2447         f"Tabular CV fold {fold_index}/{cv_folds} completed. "
2448         f"train={y_train.shape[0]}, val={y_val.shape[0]}"
2449     )
2450
2451 val_true_tensor = torch.tensor(oof_val_true, dtype=torch.long).unsqueeze(1)
2452 aggregate_margin = calculate_average_margin_of_victory(oof_val_prob, oof_reference_prices)
2453

```

```

2654     print(
2655         f"Prepared tabular data with CV ({cv_folds} folds). "
2656         f"samples={X.shape[0]}, features={len(feature_names)}"
2657     )
2658
2659     return (
2660         fold_results,
2661         all_train_true,
2662         all_train_prob,
2663         oof_val_prob,
2664         aggregate_margin,
2665         val_true_tensor,
2666     )
2667
2668
2669 def run_tabular_experiment_on_ratio(
2670     mapped_market_folder_path: Path,
2671     truncate_and_keep_ratio: float,
2672     *,
2673     preset: str,
2674     market_policy: Callable[[dict[str, Any]], bool] | None = None,
2675     val_ratio: float = 0.2,
2676     vwap_window: int = 4,
2677     whale_top_k: int = 10,
2678     seed: int = 42,
2679     use_cross_validation: bool = True,
2680     cv_folds: int = 5,
2681     roc_output_dir: Path,
2682     confusion_matrix_output_dir: Path | None = None,
2683 ) -> RunResult:
2684     """Run the tabular experiment for one truncation ratio and save its artifacts."""
2685     if confusion_matrix_output_dir is None:
2686         confusion_matrix_output_dir = roc_output_dir.parent.parent / "confusion_matrix" / "tabular"
2687
2688     X, y, feature_names = _load_tabular_dataset(
2689         mapped_market_folder_path,
2690         truncate_and_keep_ratio=truncate_and_keep_ratio,
2691         market_policy=market_policy,
2692         vwap_window=vwap_window,
2693         whale_top_k=whale_top_k,
2694     )
2695
2696     if REFERENCE_PRICE_FEATURE not in feature_names:
2697         raise ValueError(f"{REFERENCE_PRICE_FEATURE} not found in feature names.")
2698
2699     if use_cross_validation:
2700         (
2701             fold_results,
2702             y_true_train,
2703             y_prob_train,
2704             y_prob_val,
2705             average_margin_of_victory,
2706             y_true_val_tensor,
2707         ) = _run_tabular_cross_validation(
2708             X=X,
2709             y=y,
2710             feature_names=feature_names,
2711             seed=seed,
2712             cv_folds=cv_folds,
2713         )
2714         train_metrics = average_binary_metrics([fold.train_metrics for fold in fold_results])
2715         val_metrics = evaluate_binary_metrics(
2716             y_true_val_tensor,
2717             y_prob_val,
2718             classification_threshold=CLASSIFICATION_THRESHOLD,
2719         )
2720         file_suffix = f"_cv{cv_folds}"
2721         cv_metadata = (True, cv_folds)
2722     else:
2723         (
2724             fold_results,
2725             y_true_train,
2726             y_prob_train,
2727             y_prob_val,

```

```

2728         y_true_val_tensor,
2729         average_margin_of_victory,
2730     ) = _run_tabular_single_split(
2731         X=X,
2732         y=y,
2733         feature_names=feature_names,
2734         val_ratio=val_ratio,
2735         seed=seed,
2736     )
2737     train_metrics = fold_results[0].train_metrics
2738     val_metrics = fold_results[0].val_metrics
2739     file_suffix = ""
2740     cv_metadata = (False, None)
2741
2742     roc_file_name = f"{preset}_ratio_{truncate_and_keep_ratio:.2f}{file_suffix}.png"
2743     roc_plot_path = save_roc_plot(
2744         output_path=roc_output_dir / roc_file_name,
2745         title=(
2746             f"Tabular ROC (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2747             f"mode={'cv' if use_cross_validation else 'single'})"
2748         ),
2749         roc_fpr=val_metrics.roc_fpr,
2750         roc_tpr=val_metrics.roc_tpr,
2751         auc=val_metrics.auc,
2752         dpi=300,
2753     )
2754
2755     y_pred_train = [1 if probability >= CLASSIFICATION_THRESHOLD else 0 for probability in y_prob_train]
2756     y_true_train_list = [int(value) for value in y_true_train]
2757
2758     y_true_val_list = [int(value) for value in y_true_val_tensor.detach().cpu().squeeze(1).tolist()]
2759     y_pred_val = [1 if probability >= CLASSIFICATION_THRESHOLD else 0 for probability in y_prob_val]
2760
2761     train_confusion_matrix_file_name = (
2762         f"{preset}_ratio_{truncate_and_keep_ratio:.2f}{file_suffix}_train.png"
2763     )
2764     val_confusion_matrix_file_name = (
2765         f"{preset}_ratio_{truncate_and_keep_ratio:.2f}{file_suffix}_val.png"
2766     )
2767     train_confusion_matrix_plot_path = save_confusion_matrix_plot(
2768         output_path=confusion_matrix_output_dir / train_confusion_matrix_file_name,
2769         title=(
2770             f"Tabular Train Confusion Matrix (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2771             f"mode={'cv' if use_cross_validation else 'single'})"
2772         ),
2773         y_true=y_true_train_list,
2774         y_pred=y_pred_train,
2775         dpi=300,
2776     )
2777     val_confusion_matrix_plot_path = save_confusion_matrix_plot(
2778         output_path=confusion_matrix_output_dir / val_confusion_matrix_file_name,
2779         title=(
2780             f"Tabular Val Confusion Matrix (preset={preset}, ratio={truncate_and_keep_ratio:.2f}, "
2781             f"mode={'cv' if use_cross_validation else 'single'})"
2782         ),
2783         y_true=y_true_val_list,
2784         y_pred=y_pred_val,
2785         dpi=300,
2786     )
2787
2788     cv_enabled, run_cv_folds = cv_metadata
2789     return RunResult(
2790         preset=preset,
2791         strategy="tabular",
2792         truncate_and_keep_ratio=truncate_and_keep_ratio,
2793         desired_num_candlesticks=None,
2794         cv_enabled=cv_enabled,
2795         cv_folds=run_cv_folds,
2796         train_metrics=train_metrics,
2797         val_metrics=val_metrics,
2798         average_margin_of_victory=average_margin_of_victory,
2799         fold_results=fold_results,
2800         roc_plot_path=roc_plot_path,
2801         train_confusion_matrix_plot_path=train_confusion_matrix_plot_path,

```

```

2802     val_confusion_matrix_plot_path=val_confusion_matrix_plot_path,
2803 )

```

A.2 preprocessing Package

Listing 4: Source code for the preprocessing package.

```

1 FILE: preprocessing/__init__.py
2 =====
3 """Preprocessing pipeline package."""
4
5 __all__ = []
6
7
8
9 =====
10 FILE: preprocessing/cli.py
11 =====
12 import argparse
13 from datetime import datetime, timezone
14 from pathlib import Path
15
16 from preprocessing.config import (
17     DEFAULT_END_DATE_MAX,
18     DEFAULT_END_DATE_MIN,
19     DEFAULT_LIMIT,
20     DEFAULT_MAPPED_MARKETS_FOLDER,
21     DEFAULT_MARKETS_METADATA_PATH,
22     DEFAULT_START_DATE_MIN,
23     DEFAULT_TIMEOUT_SECONDS,
24     DEFAULT_TRADE_COUNT_THRESHOLD,
25     DEFAULT_TRADES_FOLDER,
26     FetchConfig,
27     MatchConfig,
28     VisualizeConfig,
29 )
30 from preprocessing.fetch import fetch_all_finished_markets, post_process_markets
31 from preprocessing.io import write_json
32 from preprocessing.match import run_match
33 from preprocessing.visualize import run_visualization
34
35
36 def _format_datetime(value: datetime) -> str:
37     """Format a datetime in UTC ISO-8601 'Z' notation."""
38     return value.astimezone(timezone.utc).isoformat().replace("+00:00", "Z")
39
40
41 def _parse_datetime(value: str) -> datetime:
42     """Parse an ISO-8601 datetime string and default naive values to UTC."""
43     normalized = value.replace("Z", "+00:00")
44     parsed = datetime.fromisoformat(normalized)
45     if parsed.tzinfo is None:
46         parsed = parsed.replace(tzinfo=timezone.utc)
47     return parsed
48
49
50 def _build_parser() -> argparse.ArgumentParser:
51     """Build the preprocessing pipeline CLI parser."""
52     parser = argparse.ArgumentParser(description="Preprocessing pipeline CLI.")
53     subparsers = parser.add_subparsers(dest="command", required=True)
54
55     fetch_parser = subparsers.add_parser("fetch", help="Fetch finished market metadata.")
56     fetch_parser.add_argument(
57         "--start-date-min",
58         type=_parse_datetime,
59         default=DEFAULT_START_DATE_MIN,
60         help=f"Min market start date (default: {_format_datetime(DEFAULT_START_DATE_MIN)}).",
61     )
62     fetch_parser.add_argument(
63         "--end-date-min",
64         type=_parse_datetime,
65         default=DEFAULT_END_DATE_MIN,
66         help=f"Min market end date (default: {_format_datetime(DEFAULT_END_DATE_MIN)}).",

```

```

67 )
68 fetch_parser.add_argument(
69     "--end-date-max",
70     type=_parse_datetime,
71     default=DEFAULT_END_DATE_MAX,
72     help=f"Max market end date (default: {_format_datetime(DEFAULT_END_DATE_MAX)}).",
73 )
74 fetch_parser.add_argument(
75     "--limit",
76     type=int,
77     default=DEFAULT_LIMIT,
78     help=f"Page size for pagination (default: {DEFAULT_LIMIT}).",
79 )
80 fetch_parser.add_argument(
81     "--timeout-seconds",
82     type=int,
83     default=DEFAULT_TIMEOUT_SECONDS,
84     help=f"HTTP timeout seconds (default: {DEFAULT_TIMEOUT_SECONDS}).",
85 )
86 fetch_parser.add_argument(
87     "--output-path",
88     type=Path,
89     default=DEFAULT_MARKETS_METADATA_PATH,
90     help=f"Output path for market metadata JSON (default: {DEFAULT_MARKETS_METADATA_PATH}).",
91 )
92
93 match_parser = subparsers.add_parser("match", help="Map trades into each market folder.")
94 match_parser.add_argument(
95     "--metadata-path",
96     type=Path,
97     default=DEFAULT_MARKETS_METADATA_PATH,
98     help=f"Path to all market metadata JSON (default: {DEFAULT_MARKETS_METADATA_PATH}).",
99 )
100 match_parser.add_argument(
101     "--trades-folder",
102     type=Path,
103     default=DEFAULT_TRADES_FOLDER,
104     help=f"Folder containing trade CSV files (default: {DEFAULT_TRADES_FOLDER}).",
105 )
106 match_parser.add_argument(
107     "--output-folder",
108     type=Path,
109     default=DEFAULT_MAPPED_MARKETS_FOLDER,
110     help=f"Output mapped markets folder (default: {DEFAULT_MAPPED_MARKETS_FOLDER}).",
111 )
112 match_parser.add_argument(
113     "--trade-count-threshold",
114     type=int,
115     default=DEFAULT_TRADE_COUNT_THRESHOLD,
116     help=(
117         "Keep markets whose trade_count is strictly greater than this threshold "
118         f"(default: {DEFAULT_TRADE_COUNT_THRESHOLD})."
119     ),
120 )
121 match_parser.add_argument(
122     "--no-clean-output-folder",
123     action="store_true",
124     help="Do not clear output folder before writing mapped markets.",
125 )
126
127 visualize_parser = subparsers.add_parser(
128     "visualize",
129     help="Generate per-market price history and trade-count distribution plots.",
130 )
131 visualize_parser.add_argument(
132     "--mapped-markets-folder",
133     type=Path,
134     default=DEFAULT_MAPPED_MARKETS_FOLDER,
135     help=f"Mapped markets folder (default: {DEFAULT_MAPPED_MARKETS_FOLDER}).",
136 )
137 visualize_parser.add_argument(
138     "--visualization-output-folder",
139     type=Path,
140     default=DEFAULT_MAPPED_MARKETS_FOLDER,

```

```

41     help=(
42         "Folder for visualization outputs "
43         f"(default: {DEFAULT_MAPPED_MARKETS_FOLDER})."
44     ),
45 )
46
47 prepare_parser = subparsers.add_parser(
48     "prepare",
49     help="Run full pipeline: fetch -> match -> visualize.",
50 )
51 prepare_parser.add_argument(
52     "--start-date-min",
53     type=_parse_datetime,
54     default=DEFAULT_START_DATE_MIN,
55     help=f"Min market start date (default: {_format_datetime(DEFAULT_START_DATE_MIN)}).",
56 )
57 prepare_parser.add_argument(
58     "--end-date-min",
59     type=_parse_datetime,
60     default=DEFAULT_END_DATE_MIN,
61     help=f"Min market end date (default: {_format_datetime(DEFAULT_END_DATE_MIN)}).",
62 )
63 prepare_parser.add_argument(
64     "--end-date-max",
65     type=_parse_datetime,
66     default=DEFAULT_END_DATE_MAX,
67     help=f"Max market end date (default: {_format_datetime(DEFAULT_END_DATE_MAX)}).",
68 )
69 prepare_parser.add_argument(
70     "--limit",
71     type=int,
72     default=DEFAULT_LIMIT,
73     help=f"Page size for pagination (default: {DEFAULT_LIMIT}).",
74 )
75 prepare_parser.add_argument(
76     "--timeout-seconds",
77     type=int,
78     default=DEFAULT_TIMEOUT_SECONDS,
79     help=f"HTTP timeout seconds (default: {DEFAULT_TIMEOUT_SECONDS}).",
80 )
81 prepare_parser.add_argument(
82     "--metadata-path",
83     type=Path,
84     default=DEFAULT_MARKETS_METADATA_PATH,
85     help=f"Output path for market metadata JSON (default: {DEFAULT_MARKETS_METADATA_PATH}).",
86 )
87 prepare_parser.add_argument(
88     "--trades-folder",
89     type=Path,
90     default=DEFAULT_TRADES_FOLDER,
91     help=f"Folder containing trade CSV files (default: {DEFAULT_TRADES_FOLDER}).",
92 )
93 prepare_parser.add_argument(
94     "--output-folder",
95     type=Path,
96     default=DEFAULT_MAPPED_MARKETS_FOLDER,
97     help=f"Output mapped markets folder (default: {DEFAULT_MAPPED_MARKETS_FOLDER}).",
98 )
99 prepare_parser.add_argument(
100     "--trade-count-threshold",
101     type=int,
102     default=DEFAULT_TRADE_COUNT_THRESHOLD,
103     help=(
104         "Keep markets whose trade_count is strictly greater than this threshold "
105         f"(default: {DEFAULT_TRADE_COUNT_THRESHOLD})."
106     ),
107 )
108 prepare_parser.add_argument(
109     "--no-clean-output-folder",
110     action="store_true",
111     help="Do not clear output folder before writing mapped markets.",
112 )
113 prepare_parser.add_argument(
114     "--visualization-output-folder",

```

```

215     type=Path,
216     default=DEFAULT_MAPPED_MARKETS_FOLDER,
217     help=(
218         "Folder for visualization outputs "
219         f"(default: {DEFAULT_MAPPED_MARKETS_FOLDER})."
220     ),
221 )
222
223 return parser
224
225
226 def _run_fetch(config: FetchConfig) -> None:
227     """Fetch, post-process, and persist finished market metadata."""
228     markets = fetch_all_finished_markets(config)
229     markets = post_process_markets(markets)
230     write_json(config.output_path, markets)
231     print(
232         f"Successfully fetched {len(markets)} finished markets and "
233         f"saved to {config.output_path}"
234     )
235
236
237 def _run_match(config: MatchConfig) -> None:
238     """Run the trade-to-market matching pipeline."""
239     run_match(config)
240
241
242 def _run_visualize(config: VisualizeConfig) -> None:
243     """Run the mapped-market visualization pipeline."""
244     run_visualization(config)
245
246
247 def main() -> None:
248     """Run the preprocessing CLI and dispatch the selected subcommand."""
249     args = _build_parser().parse_args()
250
251     if args.command == "fetch":
252         _run_fetch(
253             FetchConfig(
254                 start_date_min=args.start_date_min,
255                 end_date_min=args.end_date_min,
256                 end_date_max=args.end_date_max,
257                 limit=args.limit,
258                 timeout_seconds=args.timeout_seconds,
259                 output_path=args.output_path,
260             )
261         )
262         return
263
264     if args.command == "match":
265         _run_match(
266             MatchConfig(
267                 metadata_path=args.metadata_path,
268                 trades_folder=args.trades_folder,
269                 output_folder=args.output_folder,
270                 trade_count_threshold=args.trade_count_threshold,
271                 clean_output_folder=not args.no_clean_output_folder,
272             )
273         )
274         return
275
276     if args.command == "visualize":
277         _run_visualize(
278             VisualizeConfig(
279                 mapped_markets_folder_path=args.mapped_markets_folder,
280                 visualization_output_folder_path=args.visualization_output_folder,
281             )
282         )
283         return
284
285     if args.command == "prepare":
286         fetch_config = FetchConfig(
287             start_date_min=args.start_date_min,
288             end_date_min=args.end_date_min,

```

```

389         end_date_max=args.end_date_max,
390         limit=args.limit,
391         timeout_seconds=args.timeout_seconds,
392         output_path=args.metadata_path,
393     )
394     match_config = MatchConfig(
395         metadata_path=args.metadata_path,
396         trades_folder=args.trades_folder,
397         output_folder=args.output_folder,
398         trade_count_threshold=args.trade_count_threshold,
399         clean_output_folder=not args.no_clean_output_folder,
400     )
401     visualize_config = VisualizeConfig(
402         mapped_markets_folder_path=args.output_folder,
403         visualization_output_folder_path=args.visualization_output_folder,
404     )
405
406     print("[prepare] Step 1/3: fetch")
407     _run_fetch(fetch_config)
408     print("[prepare] Step 2/3: match")
409     _run_match(match_config)
410     print("[prepare] Step 3/3: visualize")
411     _run_visualize(visualize_config)
412     return
413
414     raise ValueError(f"Unsupported command: {args.command}")
415
416
417 if __name__ == "__main__":
418     main()
419
420
421
422 =====
423 FILE: preprocessing/config.py
424 =====
425 from dataclasses import dataclass
426 from datetime import datetime
427 from pathlib import Path
428
429 DEFAULT_START_DATE_MIN = datetime.fromisoformat("2025-12-25T00:00:00+00:00")
430 DEFAULT_END_DATE_MIN = datetime.fromisoformat("2025-12-25T00:00:00+00:00")
431 DEFAULT_END_DATE_MAX = datetime.fromisoformat("2025-12-31T23:59:59+00:00")
432
433 DEFAULT_MARKETS_METADATA_PATH = Path("datasets/metadata/all_markets_metadata.json")
434 DEFAULT_TRADES_FOLDER = Path("datasets/trades")
435 DEFAULT_MAPPED_MARKETS_FOLDER = Path("datasets/mapped_markets")
436
437 DEFAULT_LIMIT = 500
438 DEFAULT_TIMEOUT_SECONDS = 30
439 DEFAULT_TRADE_COUNT_THRESHOLD = 1
440
441 TRADE_HEADERS = [
442     "block_number",
443     "timestamp",
444     "tx_hash",
445     "wallet",
446     "side",
447     "tokens",
448     "price",
449     "total_usdc",
450 ]
451
452
453 @dataclass(frozen=True)
454 class FetchConfig:
455     start_date_min: datetime = DEFAULT_START_DATE_MIN
456     end_date_min: datetime = DEFAULT_END_DATE_MIN
457     end_date_max: datetime = DEFAULT_END_DATE_MAX
458     limit: int = DEFAULT_LIMIT
459     timeout_seconds: int = DEFAULT_TIMEOUT_SECONDS
460     output_path: Path = DEFAULT_MARKETS_METADATA_PATH
461
462

```

```

363 @dataclass(frozen=True)
364 class MatchConfig:
365     metadata_path: Path = DEFAULT_MARKETS_METADATA_PATH
366     trades_folder: Path = DEFAULT_TRADES_FOLDER
367     output_folder: Path = DEFAULT_MAPPED_MARKETS_FOLDER
368     trade_count_threshold: int = DEFAULT_TRADE_COUNT_THRESHOLD
369     clean_output_folder: bool = True
370
371
372 @dataclass(frozen=True)
373 class VisualizeConfig:
374     mapped_markets_folder_path: Path = DEFAULT_MAPPED_MARKETS_FOLDER
375     visualization_output_folder_path: Path = DEFAULT_MAPPED_MARKETS_FOLDER
376
377
378
379 =====
380 FILE: preprocessing/fetch.py
381 =====
382 import json
383 from datetime import datetime, timezone
384 from typing import Any
385
386 import requests
387
388 from preprocessing.config import FetchConfig
389
390 GAMMA_BASE_URL = "https://gamma-api.polymarket.com"
391
392
393 def _to_iso_z(value: datetime) -> str:
394     """Format a datetime in UTC ISO-8601 'Z' notation."""
395     return value.astimezone(timezone.utc).isoformat().replace("+00:00", "Z")
396
397
398 def _parse_iso_datetime(value: str) -> datetime:
399     """Parse an ISO-8601 datetime string that may end with 'Z'."""
400     return datetime.fromisoformat(value.replace("Z", "+00:00"))
401
402
403 def fetch_all_finished_markets(config: FetchConfig) -> list[dict[str, Any]]:
404     """Fetch all finished markets that satisfy the configured date filters."""
405     if config.limit <= 0:
406         raise ValueError("limit must be > 0")
407
408     end_date_min_iso = _to_iso_z(config.end_date_min)
409     end_date_max_iso = _to_iso_z(config.end_date_max)
410
411     markets: list[dict[str, Any]] = []
412     offset = 0
413
414     session = requests.Session()
415
416     while True:
417         response = session.get(
418             f"{GAMMA_BASE_URL}/markets",
419             params={
420                 "closed": "true",
421                 "limit": config.limit,
422                 "offset": offset,
423                 "end_date_min": end_date_min_iso,
424                 "end_date_max": end_date_max_iso,
425             },
426             timeout=config.timeout_seconds,
427         )
428         response.raise_for_status()
429
430         page = response.json()
431         if not isinstance(page, list):
432             raise TypeError("Unexpected response: expected a list of markets")
433         if not page:
434             break
435
436         filtered_page = [

```

```

437         market
438         for market in page
439             if _parse_iso_datetime(str(market["startDate"])) > config.start_date_min
440         ]
441     markets.extend(filtered_page)
442
443     print(
444         f"fetched: {len(page)} markets and keep: {len(filtered_page)} "
445         f"from offset={offset}, total keep: {len(markets)}"
446     )
447     offset += len(page)
448
449     return markets
450
451
452 def post_process_markets(markets: list[dict[str, Any]]) -> list[dict[str, Any]]:
453     """Infer binary outcomes from market payloads and drop invalid entries."""
454     post_processed_markets: list[dict[str, Any]] = []
455
456     for market in markets:
457         outcome_prices = json.loads(str(market["outcomePrices"]))
458         if not isinstance(outcome_prices, list) or len(outcome_prices) < 2:
459             print(
460                 f"Unexpected outcomePrices format for market id: {market['id']}. "
461                 "Skipping this market."
462             )
463             continue
464
465         first = str(outcome_prices[0])
466         second = str(outcome_prices[1])
467
468         if first == "0" and second == "1":
469             market["outcome"] = "no"
470         elif first == "1" and second == "0":
471             market["outcome"] = "yes"
472         else:
473             print(
474                 f"Unexpected outcomePrices: {outcome_prices} "
475                 f"for market id: {market['id']}. Skipping this market."
476             )
477             continue
478
479         post_processed_markets.append(market)
480
481     print(f"Post processed markets: {len(post_processed_markets)} / {len(markets)}")
482     return post_processed_markets
483
484
485
486 =====
487 FILE: preprocessing/io.py
488 =====
489 import csv
490 import json
491 import shutil
492 from pathlib import Path
493 from typing import Any
494
495 from preprocessing.config import TRADE_HEADERS
496
497
498 def ensure_parent_dir(path: Path) -> None:
499     """Create the parent directory for a file path if it does not exist."""
500     path.parent.mkdir(parents=True, exist_ok=True)
501
502
503 def read_json(path: Path) -> Any:
504     """Read and return JSON data from 'path'."""
505     if not path.exists():
506         raise FileNotFoundError(f"File does not exist: {path}")
507     with open(path, "r", encoding="utf-8") as file:
508         return json.load(file)
509
510

```

```

311 def write_json(path: Path, data: Any) -> None:
312     """Write JSON data to 'path' after creating its parent directory."""
313     ensure_parent_dir(path)
314     with open(path, "w", encoding="utf-8") as file:
315         json.dump(data, file, indent=4, ensure_ascii=False)
316
317
318 def iter_trade_csv_files(trades_folder: Path) -> list[Path]:
319     """Return the sorted trade CSV files in the trades folder."""
320     if not trades_folder.exists():
321         raise FileNotFoundError(f"Trades folder does not exist: {trades_folder}")
322     return sorted(trades_folder.glob("*.csv"), key=lambda item: item.name)
323
324
325 def clear_directory(path: Path) -> None:
326     """Delete all files and subdirectories inside 'path' if it exists."""
327     if not path.exists():
328         return
329     for item in sorted(path.iterdir(), key=lambda entry: entry.name):
330         if item.is_dir():
331             shutil.rmtree(item)
332         else:
333             item.unlink()
334
335
336 def save_mapped_markets(markets: list[dict[str, Any]], output_folder: Path) -> None:
337     """Write mapped market metadata and YES/NO trade CSV files to disk."""
338     output_folder.mkdir(parents=True, exist_ok=True)
339
340     for market in markets:
341         market_id = str(market["id"])
342         market_folder = output_folder / market_id
343         market_folder.mkdir(parents=True, exist_ok=True)
344
345         yes_trades_path = market_folder / "yes.csv"
346         no_trades_path = market_folder / "no.csv"
347
348         with open(yes_trades_path, "w", encoding="utf-8", newline="") as file:
349             writer = csv.DictWriter(file, fieldnames=TRADE_HEADERS)
350             writer.writeheader()
351             writer.writerows(market["yes_trades"])
352
353         with open(no_trades_path, "w", encoding="utf-8", newline="") as file:
354             writer = csv.DictWriter(file, fieldnames=TRADE_HEADERS)
355             writer.writeheader()
356             writer.writerows(market["no_trades"])
357
358         metadata_exclude_trades = {
359             key: value
360             for key, value in market.items()
361             if key not in ["yes_trades", "no_trades"]
362         }
363         write_json(market_folder / "metadata.json", metadata_exclude_trades)
364
365
366
367 =====
368 FILE: preprocessing/match.py
369 =====
370 import csv
371 import json
372 from pathlib import Path
373 from typing import Any
374
375 from preprocessing.config import MatchConfig
376 from preprocessing.io import clear_directory, iter_trade_csv_files, read_json, save_mapped_markets
377
378 TokenToMarketSide = dict[str, tuple[dict[str, Any], str]]
379
380
381 def load_markets_metadata(metadata_path: Path) -> tuple[TokenToMarketSide, list[dict[str, Any]]]:
382     """Load market metadata and build a token-to-market-side lookup table."""
383     raw = read_json(metadata_path)
384     if not isinstance(raw, list):

```

```

385         raise TypeError(f"Expected list in metadata JSON: {metadata_path}")
386
387     markets: list[dict[str, Any]] = raw
388     print(f"Loaded {len(markets)} markets from {metadata_path}")
389
390     token_to_market_side: TokenToMarketSide = {}
391     for market in markets:
392         market["trade_count"] = 0
393         market["yes_trade_count"] = 0
394         market["no_trade_count"] = 0
395         market["yes_trades"] = []
396         market["no_trades"] = []
397
398         clob_token_ids = json.loads(str(market["clobTokenIds"]))
399         if not isinstance(clob_token_ids, list) or len(clob_token_ids) < 2:
400             raise ValueError(f"Invalid clobTokenIds for market id {market.get('id')}")
401
402         yes_token_id = str(clob_token_ids[0])
403         no_token_id = str(clob_token_ids[1])
404         token_to_market_side[yes_token_id] = (market, "yes")
405         token_to_market_side[no_token_id] = (market, "no")
406
407     return token_to_market_side, markets
408
409
410 def map_markets_and_trades(
411     token_to_market_side: TokenToMarketSide,
412     markets: list[dict[str, Any]],
413     trades_folder: Path,
414     trade_count_threshold: int,
415 ) -> list[dict[str, Any]]:
416     """Attach trades to markets and keep only markets above the trade threshold."""
417     for trade_file_path in iter_trade_csv_files(trades_folder):
418         with open(trade_file_path, mode="r", encoding="utf-8") as csvfile:
419             reader = csv.DictReader(csvfile)
420             for row in reader:
421                 token_id = str(row.get("token_id", ""))
422                 hit = token_to_market_side.get(token_id)
423                 if hit is None:
424                     continue
425
426                 market, side_key = hit
427                 trade_info = {
428                     "block_number": row.get("block_number", ""),
429                     "timestamp": row.get("timestamp", ""),
430                     "tx_hash": row.get("tx_hash", ""),
431                     "wallet": row.get("wallet", ""),
432                     "side": row.get("side", ""),
433                     "tokens": row.get("tokens", ""),
434                     "price": row.get("price", ""),
435                     "total_usdc": row.get("total_usdc", ""),
436                 }
437
438                 if side_key == "yes":
439                     market["yes_trades"].append(trade_info)
440                     market["yes_trade_count"] += 1
441                 else:
442                     market["no_trades"].append(trade_info)
443                     market["no_trade_count"] += 1
444
445                 market["trade_count"] += 1
446
447     filtered_mapped_markets = [
448         market for market in markets if int(market["trade_count"]) > trade_count_threshold
449     ]
450     print(
451         f"Found {len(filtered_mapped_markets)} / {len(markets)} markets "
452         f"with at least {trade_count_threshold} trade."
453     )
454
455     for market in filtered_mapped_markets:
456         market["yes_trades"].sort(key=lambda trade: str(trade["timestamp"]))
457         market["no_trades"].sort(key=lambda trade: str(trade["timestamp"]))
458

```

```

659     return filtered_mapped_markets
660
661
662 def run_match(config: MatchConfig) -> list[dict[str, Any]]:
663     """Run market/trade matching and persist mapped market folders."""
664     if config.clean_output_folder:
665         clear_directory(config.output_folder)
666
667     token_to_market_side, markets = load_markets_metadata(config.metadata_path)
668     filtered_mapped_markets = map_markets_and_trades(
669         token_to_market_side,
670         markets,
671         config.trades_folder,
672         config.trade_count_threshold,
673     )
674     save_mapped_markets(filtered_mapped_markets, output_folder=config.output_folder)
675     print(
676         f"Successfully saved trades for {len(filtered_mapped_markets)} markets "
677         f"to {config.output_folder}"
678     )
679     return filtered_mapped_markets
680
681
682
683 =====
684 FILE: preprocessing/visualize.py
685 =====
686 from pathlib import Path
687
688 import matplotlib.pyplot as plt
689
690 from analysis_and_experiments.data import load_market_and_trades, market_filter_policy
691 from preprocessing.config import VisualizeConfig
692
693
694 def _visualize_yes_price_history_for_market(
695     market: dict[str, object],
696     trades: list[dict[str, object]],
697     output_dir: Path,
698 ) -> None:
699     """Save a YES-price history plot for a single market."""
700     output_dir.mkdir(parents=True, exist_ok=True)
701     prices = [float(trade["price"]) for trade in trades]
702     trade_index = list(range(len(prices)))
703
704     plt.figure(figsize=(10, 6))
705     plt.plot(trade_index, prices, label="Yes Trades", marker="o")
706     plt.xlabel("Timestamp")
707     plt.ylabel("Price")
708     plt.title(f"Price History for Market: {market['id']}")
709     plt.legend()
710     plt.xticks(rotation=45)
711     plt.tight_layout()
712     plt.savefig(output_dir / "yes_price_history_final.png")
713     plt.close()
714
715
716 def _visualize_yes_trades_histogram(trade_count_statistics: list[int], output_dir: Path) -> None:
717     """Save a histogram of filtered YES-trade counts across markets."""
718     output_dir.mkdir(parents=True, exist_ok=True)
719     plt.figure(figsize=(10, 6))
720     plt.hist(trade_count_statistics, bins=20, edgecolor="black")
721     plt.xlabel("Number of Yes Trades (after filtering)")
722     plt.ylabel("Number of Markets")
723     plt.title("Distribution of Yes Trade Counts Across Markets")
724     plt.tight_layout()
725     plt.savefig(output_dir / "yes_trade_count_distribution.png", dpi=300)
726     plt.close()
727
728
729 def run_visualization(config: VisualizeConfig) -> None:
730     """Generate per-market price plots and an aggregate trade-count histogram."""
731     loaded_markets: list[dict[str, object]] = []
732     loaded_market_id_to_trades: dict[str, list[dict[str, object]]] = {}

```

```

733 loaded_market_id_to_folder: dict[str, Path] = {}
734
735 for market_folder in sorted(config.mapped_markets_folder_path.iterdir(), key=lambda item: item.name):
736     if not market_folder.is_dir():
737         continue
738
739     market, trades = load_market_and_trades(market_folder, truncate_and_keep_ratio=1.0)
740     if not market_filter_policy(market):
741         continue
742
743     market_id = str(market["id"])
744     loaded_markets.append(market)
745     loaded_market_id_to_trades[market_id] = trades
746     loaded_market_id_to_folder[market_id] = config.visualization_output_folder_path / market_id
747
748 trade_count_statistics: list[int] = []
749 for market in loaded_markets:
750     market_id = str(market["id"])
751     trades = loaded_market_id_to_trades[market_id]
752     market_folder = loaded_market_id_to_folder[market_id]
753     _visualize_yes_price_history_for_market(market, trades, market_folder)
754     trade_count_statistics.append(len(trades))
755
756 _visualize_yes_trades_histogram(
757     trade_count_statistics,
758     config.visualization_output_folder_path,
759 )

```